Catch-22s of reservoir computing

Yuanzhao Zhang [1] and Sean P. Cornelius [2]
[1]*Santa Fe Institute, 1399 Hyde Park Road, Santa Fe, New Mexico 87501, USA*
[2]*Department of Physics, Toronto Metropolitan University, Toronto, Ontario M5B 2K3, Canada*

(Received 19 March 2023; accepted 2 August 2023; published 25 September 2023)

Reservoir computing (RC) is a simple and efficient model-free framework for forecasting the behavior of nonlinear dynamical systems from data. Here, we show that there exist commonly-studied systems for which leading RC frameworks struggle to learn the dynamics unless key information about the underlying system is already known. We focus on the important problem of basin prediction—determining which attractor a system will converge to from its initial conditions. First, we show that the predictions of standard RC models (echo state networks) depend critically on warm-up time, requiring a warm-up trajectory containing almost the entire transient in order to identify the correct attractor. Accordingly, we turn to next-generation reservoir computing (NGRC), an attractive variant of RC that requires negligible warm-up time. By incorporating the exact nonlinearities in the original equations, we show that NGRC can accurately reconstruct intricate and high-dimensional basins of attraction, even with sparse training data (e.g., a single transient trajectory). Yet, a tiny uncertainty in the exact nonlinearity can render prediction accuracy no better than chance. Our results highlight the challenges faced by data-driven methods in learning the dynamics of multistable systems and suggest potential avenues to make these approaches more robust.

DOI: 10.1103/PhysRevResearch.5.033213

I. INTRODUCTION

Reservoir computing (RC) [1–12] is a machine learning framework for time-series predictions based on recurrent neural networks. Because only the output layer needs to be modified, RC is extremely efficient to train. Despite its simplicity, recent studies have shown that RC can be extremely powerful when it comes to learning unknown dynamical systems from data [13]. Specifically, RC has been used to reconstruct attractors [14,15], calculate Lyapunov exponents [16], infer bifurcation diagrams [17], and even predict the basins of unseen attractors [18,19]. These advances open the possibilities of using RC to improve climate modeling [20], create digital twins [21], anticipate synchronization [22,23], predict tipping points [24,25], and infer network connections [26].

Since the landmark paper demonstrating RC's ability to predict spatiotemporally chaotic systems from data [13], there has been a flurry of efforts to understand the success as well as identify limitations of RC [27–36]. As a result, more sophisticated architectures have been developed to extend the capability of the original framework, such as hybrid [37], parallel [38,39], and symmetry-aware [40] RC schemes.

One particularly promising variant of RC was proposed in 2021 and named next-generation reservoir computing (NGRC) [41]. There, instead of having a nonlinear reservoir and a linear output layer, one has a linear reservoir and a nonlinear output layer [42]. These differences, although subtle, confer several advantages: First, NGRC requires no random matrices and thus has much fewer hyperparameters that need to be optimized. Moreover, each NGRC prediction needs exceedingly few data points to initiate (as opposed to thousands of data points in standard RC), which is especially useful when predicting the basins of attraction in multistable dynamical system [43].

Understanding the basin structure is of fundamental importance for dynamical systems with multiple attractors. Such systems include neural networks [44,45], gene regulatory networks [46,47], differentiating cells [48,49], and power grids [50,51]. Basins of attraction provide a mapping from initial conditions to attractors and, in the face of noise or perturbations, tell us the robustness of each stable state. Despite their importance, basins have not been well studied from a machine learning perspective, with most methods for data-driven modeling of dynamical systems currently focusing on systems with a single attractor.

In this article, we show that the success of standard RC in predicting the dynamics of multistable systems can depend critically on having access to long initialization trajectories, while the performance of NGRC can be extremely sensitive to the choice of readout nonlinearity. It has been observed that for each new initial condition, a standard RC model needs to be "warmed up" with thousands of data points before it can start making predictions [43]. In practice, such data will not exist for most initial conditions. Even when they do exist, we demonstrate that the warm-up time series would often have already approached the attractor, rendering

predictions unnecessary [52]. In contrast, NGRC can easily reproduce highly intermingled and high-dimensional basins with minimal warm-up, provided the exact nonlinearity in the underlying equations is known. However, a 1% uncertainty on that nonlinearity can already make the NGRC basin predictions barely outperform random guesses. Given this extreme sensitivity, even if one had partial (but imprecise) knowledge of the underlying system, a hybrid scheme combining NGRC and such knowledge would still struggle in making reliable predictions.

The rest of the paper is organized as follows. In Sec. II, we introduce the first model system under study—the magnetic pendulum, which is representative of the difficulties of basin prediction in real nonlinear systems. In Secs. III–IV, we apply standard RC to this system, showing that accurate predictions rely heavily on the length of the warm-up trajectory. We thus turn to next-generation reservoir computing, giving a brief overview of its implementation in Sec. V. We present our main results in Sec. VI, where we characterize the effect of readout nonlinearity on NGRC's ability to predict the basins of the magnetic pendulum. We further support our findings using coupled Kuramoto oscillators in Sec. VII, which can have a large number of coexisting high-dimensional basins. Finally, we discuss the implications of our results and suggest avenues for future research in Sec. VIII.

II. THE MAGNETIC PENDULUM

For concreteness, we focus on the magnetic pendulum [53] as a representative model. It is mechanistically simple—being low-dimensional and generated by simple physical laws—and yet captures all characteristics of the basin prediction problem in general: The system is multistable and predicting which attractor a given initial condition will go to is nontrivial.

The system consists of an iron bob suspended by a massless rod above three identical magnets, located at the vertices of an equilateral triangle in the (x, y) plane (Fig. 1). The bob moves under the influence of gravity, drag due to air friction, and the attractive forces of the magnets. For simplicity, we treat the magnets as magnetic point charges and assume that the length of the pendulum rod is much greater than the distance between the magnets, allowing us to describe the dynamics using a small-angle approximation.

The resulting dimensionless equations of motion for the pendulum bob are

$$\ddot{x} = -\omega_0^2 x - a\dot{x} + \sum_{i=1}^{3} \frac{\tilde{x}_i - x}{D(\tilde{x}_i, \tilde{y}_i)^3}, \tag{1}$$

$$\ddot{y} = -\omega_0^2 y - a\dot{y} + \sum_{i=1}^{3} \frac{\tilde{y}_i - y}{D(\tilde{x}_i, \tilde{y}_i)^3}, \tag{2}$$

where $(\tilde{x}_i, \tilde{y}_i)$ are the coordinates of the ith magnet, ω_0 is the pendulum's natural frequency, and a is the damping coefficient. Here, $D(\tilde{x}, \tilde{y})$ denotes the distance between the bob and a given point $(\tilde{x}, \tilde{y})$ in the magnets' plane,

$$D(\tilde{x}, \tilde{y}) = \sqrt{(\tilde{x} - x)^2 + (\tilde{y} - y)^2 + h^2}, \tag{3}$$

where h is the bob's height above the plane. The system's four-dimensional state is thus $\mathbf{x} = (x, y, \dot{x}, \dot{y})^T$.

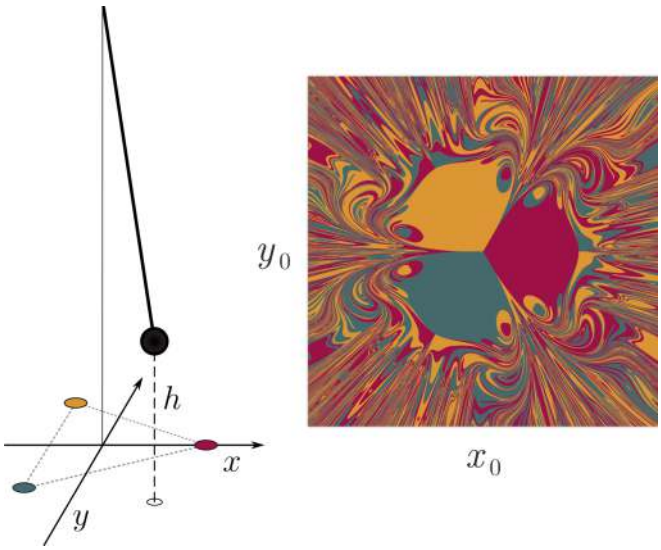

FIG. 1. Magnetic pendulum with three fixed-point attractors and the corresponding basins of attraction. (Left) Illustration of the magnetic pendulum system. Three magnets are placed on a flat surface, each drawn in the color we use to denote the corresponding basin of attraction. The hollow circle indicates the (x, y) coordinates of the pendulum bob, which together with the velocity $(\dot{x}, \dot{y})$ fully specify the system's state. (Right) Basins of attraction for the region of initial conditions under study, namely states of zero initial velocity with $-1.5 \leqslant x_0, y_0 \leqslant 1.5$.

We take the (x, y) coordinates of the magnets to be $(1/\sqrt{3}, 0)$, $(-1/2\sqrt{3}, -1/2)$, and $(-1/2\sqrt{3}, 1/2)$. Unless stated otherwise, we set $\omega_0 = 0.5$, $a = 0.2$, and $h = 0.2$ in our simulations. These values are representative of all cases for which the magnetic pendulum has exactly three stable fixed points, corresponding to the bob being at rest and pointed toward one of the three magnets.

Previous studies have largely focused on chaotic dynamics as a stress test of RC's capabilities [2,6,8,9,13,16,17,25,41]. Here we take a different approach. With nonzero damping, the magnetic pendulum dynamics is autonomous and dissipative, meaning *all* trajectories must eventually converge to a fixed point. Except on a set of initial conditions of measure zero, this will be one of the three stable fixed points identified earlier. Yet predicting which attractor a given initial condition will go to can be far from straightforward, with the pendulum wandering in an erratic transient before eventually settling to one of the three magnets [53]. This manifests as complicated basins of attraction with a "pseudo" (fat) fractal structure (Fig. 1). We can control the "fractalness" of the basins by, for example, varying the height of the pendulum h. This generates basins with tunable complexity to test the performance of (NG)RC.

III. IMPLEMENTATION OF STANDARD RC

Consider a dynamical system whose n-dimensional state $\mathbf{x}$ obeys a set of n autonomous differential equations of the form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}). \tag{4}$$

In general, the goal of reservoir computing is to approximate the flow of Eq. (4) in discrete time by a map of the form

$$\mathbf{x}_{t+1} = \mathbf{F}(\mathbf{x}_t). \tag{5}$$

Here, the index t runs over a set of discrete times separated by Δt time units of the real system, where Δt is a timescale hyperparameter generally chosen to be smaller than the characteristic timescale(s) of Eq. (4).

In standard RC, one views the state of the real system as a linear readout from an auxiliary *reservoir system*, whose state is an N_r-dimensional vector $\mathbf{r}_t$. Specifically,

$$\mathbf{x}_t = \mathbf{W} \cdot \mathbf{r}_t, \tag{6}$$

where $\mathbf{W}$ is an $n \times N_r$ matrix of trainable output weights. The reservoir system is generally much higher dimensional ($N_r \gg n$), and its dynamics obey

$$\mathbf{r}_{t+1} = (1-\alpha)\mathbf{r}_t + \alpha f(\mathbf{W}_r \cdot \mathbf{r}_t + \mathbf{W}_{\text{in}} \cdot \mathbf{u}_t + \mathbf{b}). \tag{7}$$

Here $\mathbf{W}_r$ is the $N_r \times N_r$ reservoir matrix, $\mathbf{W}_{\text{in}}$ is the $N_r \times n$ input matrix, and $\mathbf{b}$ is an N_r-dimensional bias vector. The input $\mathbf{u}_t$ is an n-dimensional vector that represents either a state of the real system ($\mathbf{u}_t = \mathbf{x}_t$) during training or the model's own output ($\mathbf{u}_t = \mathbf{W} \cdot \mathbf{r}_t$) during prediction. The nonlinear activation function f is applied elementwise, where we adopt the standard choice of $f(\cdot) = \tanh(\cdot)$. Finally, $0 < \alpha \leqslant 1$ is the so-called *leaky coefficient*, which controls the inertia of the reservoir dynamics.

In general, only the output matrix $\mathbf{W}$ is trained, with $\mathbf{W}_r$, $\mathbf{W}_{\text{in}}$, and $\mathbf{b}$ generated randomly from appropriate ensembles. We follow best practices [54] and previous studies in generating these latter components, specifically:

(i) $\mathbf{W}_r$ is the weighted adjacency matrix of a directed Erdős-Rényi graph on N_r nodes. The link probability is $0 < q \leqslant 1$, and we allow for self-loops. We first draw the link weights uniformly and independently from $[-1, 1]$, and then normalize them so that $\mathbf{W}_r$ has a specified spectral radius $\rho > 0$. Here, q and ρ are hyperparameters.

(ii) $\mathbf{W}_{\text{in}}$ is a dense matrix, whose entries are initially drawn uniformly and independently from $[-1, 1]$. In the magnetic pendulum, the state $\mathbf{x}_t$ [and hence the input term $\mathbf{u}_t$ in Eq. (7)] is of the form $(x, y, \dot{x}, \dot{y})^T$. To allow for different characteristic scales of the position vs. velocity dynamics, we scale the first two columns of $\mathbf{W}_{\text{in}}$ by s_x and the last two columns by s_v, where $s_x, s_v > 0$ are scale hyperparameters.

(iii) $\mathbf{b}$ has its entries drawn uniformly and independently from $[-s_b, s_b]$, where $s_b > 0$ is a scale hyperparameter.

Training. To train an RC model from a given initial condition $\mathbf{x}_0$, we first integrate the real dynamics (4) to obtain N_{train} additional states $\{\mathbf{x}_t\}_{t=1,\ldots,N_{\text{train}}}$. We then iterate the reservoir dynamics (7) for N_{train} times from $\mathbf{r}_0 = \mathbf{0}$, using the training data as inputs ($\mathbf{u}_t = \mathbf{x}_t$). This produces a corresponding sequence of reservoir states, $\{\mathbf{r}_t\}_{t=1,\ldots,N_{\text{train}}}$. Finally, we solve for the output weights $\mathbf{W}$ that render Eq. (6) the best fit to the training data using Ridge regression with Tikhonov regularization,

$$\mathbf{W} = \mathbf{X}\mathbf{R}^T(\mathbf{R}\mathbf{R}^T + \lambda\mathbb{I})^{-1}. \tag{8}$$

Here, $\mathbf{X}$ ($\mathbf{R}$) is a matrix whose columns are the $\mathbf{x}_t$ ($\mathbf{r}_t$) for $t = 1, \ldots, N_{\text{train}}$, $\mathbb{I}$ is the identity matrix, and $\lambda > 0$ is a regularization coefficient that prevents ill conditioning of the weights, which can be symptomatic of overfitting the data.

Prediction. To simulate a trained RC model from a given initial condition $\mathbf{x}_0$, we first integrate the true dynamics (4) forward in time to obtain a total of $N_{\text{warm-up}} \geqslant 0$ states $\{\mathbf{x}_t\}_{t=1,\ldots,N_{\text{warm-up}}}$. During the first $N_{\text{warm-up}}$ iterations of the discrete dynamics (7), the input term comes from the real trajectory, i.e., $\mathbf{u}_t = \mathbf{x}_t$. Thereafter, we replace the input with the model's own output at the previous iteration ($\mathbf{u}_t = \mathbf{W} \cdot \mathbf{r}_t$). This creates a closed-loop system from Eq. (7), which we iterate without further input from the real system.

IV. CRITICAL DEPENDENCE OF STANDARD RC ON WARM-UP TIME

Although standard RC is extremely powerful, it is known to demand large warm-up periods ($N_{\text{warm-up}}$) in certain problems in order to be stable [18]. In principle, this could create a dilemma for the problem of basin prediction, as long warm-up trajectories from the real system will generally be unavailable for initial conditions unseen during training. And even if such data were available, the problem could be rendered moot if the required warm-up exceeds the transient period of the given initial condition [43]. Here, we systematically test RC's sensitivity to the warm-up time using the magnetic pendulum system.

Our aim is to test standard RC under the most favorable conditions. Accordingly, we will train each RC model on a *single* initial condition $\mathbf{x}_0 = (x_0, y_0, 0, 0)^T$ of the magnetic pendulum, and ask it to reproduce only the trajectory from that initial condition. Likewise, before training, we systematically optimize the RC hyperparameters for that initial condition via Bayesian optimization, seeking to minimize an objective function that combines both training and validation error. For details of this process, we refer the reader to Appendix F.

In initial tests of our optimization procedure, we found it largely insensitive to the reservoir connectivity (q), with equally good training/validation performances achievable across a range of q from 0.01 to 1. We likewise found little impact of the regularization coefficient over several orders of magnitude, with the optimizer frequently pinning λ at the supplied lower bound of 10^{-8}. Thus, in the interest of more fully exploring the most important hyperparameters, we fix $q = 0.03$ and $\lambda = 10^{-8}$. We then optimize the remaining five continuous hyperparameters (ρ, s_x, s_v, s_b, α) over the ranges specified in Table II.

Throughout this section, we set $\Delta t = 0.02$, which is smaller than the characteristic timescales of the magnetic pendulum. We train each RC model on $N_{\text{train}} = 4000$ data points of the real system starting from the given initial condition, which when paired with the chosen Δt encompass both the transient dynamics and convergence to one of the attractors. We fix the reservoir size at $N_r = 300$, and we show that larger reservoir sizes do not alter our results in Appendix G.

Figure 2 shows the performance of an ensemble of RC realizations with optimized hyperparameters for the initial condition $(x_0, y_0) = (-1.2, 0.75)$. Specifically, we show the normalized root-mean-square error (NRMSE, see Appendix C) between the real and RC-predicted trajectory as a function of warm-up time ($t_{\text{warm-up}} = N_{\text{warm-up}} \cdot \Delta t$). In

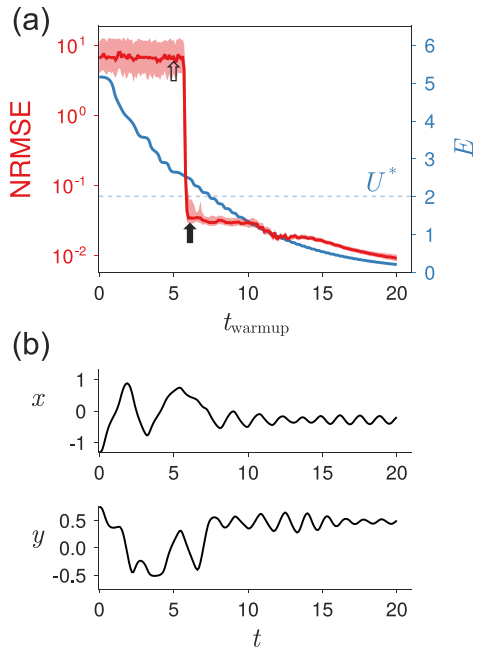

FIG. 2. Forecastability transition of standard RC. The initial condition used for both training and prediction was $(x_0, y_0) = (-1.3, 0.75)$. The optimized RC hyperparameters for this initial condition are listed in Table III. We train an ensemble of 100 different RC models with these hyperparameters, then simulate each from the training initial condition using varying numbers of warm-up points. In (a), the red line/bands denote the median/interquartile range of the resulting prediction NRMSE as a function of warm-up time. The blue curves show the total mechanical energy E of the training trajectory at the same times. We see a sharp drop in model error at $t_{\text{warm-up}} \approx 6$, only shortly before E falls below the height of the potential barriers (U^*, dashed line) separating the three wells of the magnetic pendulum. In (b), we overlay the x and y dynamics of the real system for comparison. This confirms that the "critical" warm-up time in (a) aligns closely with the end of the transient. The arrows in (a) denote the warm-up times used for the example simulations in Fig. 3.

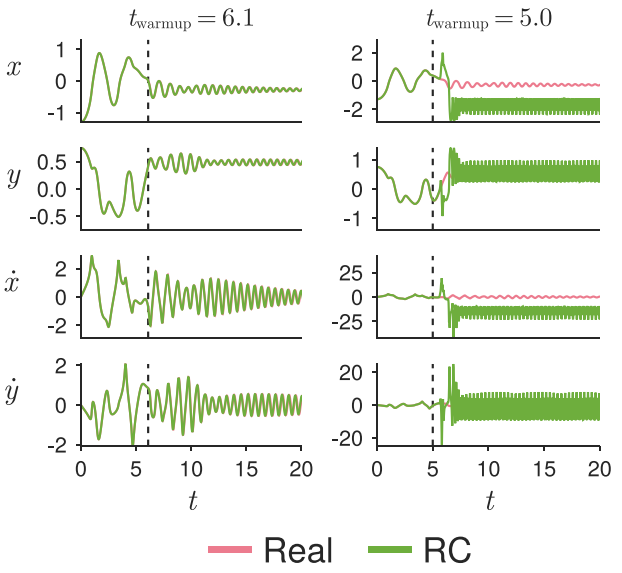

FIG. 3. Sensitivity of standard RC performance to warm-up time. We show example simulations from one RC realization in Fig. 2 with two different warm-up times (dashed lines). The initial condition and optimized RC hyperparameters are the same as in Fig. 2.

Fig. 2(a) we observe a sharp transition around $t_{\text{warm-up}} = 6$. Before this point, we consistently have NRMSE $= \mathcal{O}(1)$, meaning the RC error is comparable to the scale of the real trajectory. But after the transition, the error is always quite small (NRMSE $\ll 1$).

We can gain physical insight about this "forecastability transition" by analyzing the total mechanical energy of the training trajectory,

$$E = \tfrac{1}{2}(\dot{x}^2 + \dot{y}^2) + U(x, y). \tag{9}$$

Here $U(x, y)$ is the potential corresponding to Eqs. (1) and (2), where we set $U = 0$ at the minima corresponding to the three attractors. Strikingly, the critical warm-up time occurs only shortly before the energy drops below a critical value U^*—defined as the height of the potential barriers between the three wells [Fig. 2(a)]. By this time, the system is unambiguously "trapped" near a specific magnet, making only damped oscillations thereafter [Fig. 2(b)]. This suggests that even highly optimized RC models will fail to reproduce convergence to the correct attractor unless they have already been guided there by data from the real system.

We illustrate this further in Fig. 3, showing example predictions from one RC realization considered above under two different warm-up times: one above the critical value in Fig. 2, and one below. Indeed, with sufficient warm-up (left), the RC trajectory is a near-perfect match to the real one, both before and after the warm-up period. But if the warm-up time is even slightly less than the critical value (right), the model quickly diverges once the autonomous prediction begins. In this case, the model fails to reproduce convergence to *any* fixed-point attractor, let alone the correct one, instead oscillating wildly.

This pattern holds when we repeat our experiment for other initial conditions, re-optimizing hyperparameters and retraining an ensemble of RC models for each (Figs. 11–14 in Appendix G). In all cases, we see the same sharp drop in RC prediction error at a particular warm-up time (Figs. 11 and 13). Without at least this much warm-up time, the models fail to capture the real dynamics even qualitatively, often converging to an unphysical state with nonzero final velocity (Figs. 12 and 14). Although there exist initial conditions that require shorter warm-ups—such as $(x_0, y_0) = (1.0, -0.5)$—this is only because those initial conditions have shorter transients. Indeed, there are other initial conditions—such as $(x_0, y_0) = (1.75, 1.6)$—that have longer transients and demand commensurately larger warm-up times (Figs. 13 and 14). In no case have we observed the RC dynamics staying faithful to the real system unless the warm-up is comparable to the transient period.

Note that the breakdown of RC with insufficient warm-up time cannot be attributed to an insufficiently complex model vis-à-vis the only hyperparameter we have not optimized: the reservoir size (N_r). Indeed, we have repeated our experiment with reservoirs twice as large ($N_r = 600$). Even with optimized values of the other hyperparameters, we still see a sharp

transition in the NRMSE at a warm-up time comparable to the transient time (Fig. 15 in Appendix G).

In sum, we have shown that standard RC is unsuitable for basin prediction in this representative multistable system. Specifically, RC models can only reliably reproduce convergence to the correct attractor when they have been guided to its vicinity. This is true even with the benefit of highly tuned hyperparameters (Appendix E), and validation on only the initial condition seen during training.

For the remainder of the paper, we instead turn to next-generation reservoir computing (NGRC). Although it is known that every NGRC model implicitly defines the connectivity matrix and other parameters of a standard RC model [41,42], there is no guarantee that the two architectures would perform similarly in practice. In particular, NGRC is known to demand substantially less warm-up time [41], potentially avoiding the "catch-22" identified here for standard RC. Can this cutting-edge framework succeed in learning the magnetic pendulum and other paradigmatic multistable systems?

V. IMPLEMENTATION OF NGRC

We implement the NGRC framework following Refs. [41,43]. In NGRC, the update rule for the discrete dynamics is taken as

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{W} \cdot \mathbf{g}_t, \tag{10}$$

where $\mathbf{g}_t$ is an m-dimensional *feature vector*, calculated from the current state and $k-1$ past states, namely,

$$\mathbf{g}_t = \mathbf{g}(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-k+1}). \tag{11}$$

Here, $k \geqslant 1$ is a hyperparameter that governs the amount of memory in the NGRC model, and $\mathbf{W}$ is an $n \times m$ matrix of trainable weights.

We elaborate on the functional form of the feature embedding $\mathbf{g}$ below. But in general, the features can be divided into three groups: (i) one constant (bias) feature; (ii) $m_{\text{lin}} = nk$ linear features, corresponding to the components of $\{\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-k+1}\}$; and finally (iii) m_{nonlin} nonlinear features, each a nonlinear transformation of the linear features. The total number of features is thus $m = 1 + m_{\text{lin}} + m_{\text{nonlin}}$.

Training. Per Eq. (10), training an NGRC model amounts to finding values for the weights $\mathbf{W}$ that give the best fit for the discrete update rule

$$\mathbf{y}_t = \mathbf{W} \cdot \mathbf{g}_t, \tag{12}$$

where $\mathbf{y}_t = \mathbf{x}_{t+1} - \mathbf{x}_t$. Accordingly, we calculate pairs of inputs ($\mathbf{g}_t$) and next-step targets ($\mathbf{y}_t$) over $N_{\text{traj}} \geqslant 1$ training trajectories from the real system (4), each of length $N_{\text{train}} + k$. We then solve for the values of $\mathbf{W}$ that best fit Eq. (12) in the least-squares sense via regularized Ridge regression, namely,

$$\mathbf{W} = \mathbf{Y}\mathbf{G}^T(\mathbf{G}\mathbf{G}^T + \lambda\mathbb{I})^{-1}. \tag{13}$$

Here $\mathbf{Y}$ ($\mathbf{G}$) is a matrix whose columns are the $\mathbf{y}_t$ ($\mathbf{g}_t$). The regularization coefficient λ plays the same role as in standard RC [cf. Eq. (8)].

Prediction. To simulate a trained NGRC model from a given initial condition $\mathbf{x}_0$, we first integrate the true dynamics (4) forward in time to obtain the additional $k-1$ states needed to perform the first discrete update according to Eqs. (10) and (11). This is the warm-up period for the NGRC model. Thereafter, we iterate Eqs. (10) and (11) as an autonomous dynamical system, with each output becoming part of the model's input at the next time step. Thus in contrast to training, the model receives no data from the real system during prediction except the $k-1$ "warm-up" states.

There is a clear parallel between NGRC [41,42] and statistical forecasting methods [55] such as nonlinear vector-autoregression (NVAR). However, as noted in Ref. [56], the feature vectors of a typical NGRC model usually have far more terms than NVAR methods, as the latter was designed with interpretability in mind. It is the use of a library of many candidate features—in addition to other details like the typical training methods employed—that sets NGRC apart from classic statistical forecasting approaches. In this way, NGRC also resembles the sparse identification of nonlinear dynamics (SINDy) framework [57]. The differences here are the intended tasks (finding parsimonious models vs fitting the dynamics), the optimization schemes (LASSO vs Ridge regression), and NGRC's inclusion of delayed states (generally no delayed states for SINDy).

VI. SENSITIVE DEPENDENCE OF NGRC PERFORMANCE ON READOUT NONLINEARITY

The importance of careful feature selection is well appreciated for many machine learning frameworks [57,58]. Yet one major appeal of NGRC is that the choice of nonlinearity is considered to be of secondary importance; in many systems studied to date, one can often bypass the feature selection process by adopting some generic nonlinearities (e.g., low-order polynomials). Indeed, applications of NGRC to chaotic benchmark systems have shown good results even when the features do not include all nonlinearities in the underlying ODEs [41,59]. But can we expect this to be true in general? Here, we test NGRC's sensitivity to the choice of feature embedding $\mathbf{g}$ (i.e., readout nonlinearity) in the basin prediction problem. Specifically, we compare the performance of three candidate NGRC models, in which the nonlinearities are:

(I) *Polynomials*, specifically all unique monomials formed by the $4k$ components of $\{\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-k+1}\}$, with degree between 2 and d_{max}.

(II) As set of N_{RBF} *radial basis functions* (RBF) applied to the position coordinates $\boldsymbol{r} = (x, y)$ of each of the k states. The RBFs have randomly-chosen centers and a kernel function with shape and scale similar to the magnetic force term.

(III) The exact nonlinearities in the magnetic pendulum system. Namely, the x and y components of the magnetic force for each magnet, evaluated at each of the k states.

The details of each model are summarized in Table I. Recall that in addition to their unique nonlinear features, all models contain one constant feature (set to 1 without loss of generality) and $4k$ linear features.

Models I–III represent a hierarchy of increasing knowledge about the real system. In Model I, we assume complete ignorance, hoping that the real dynamics are well approximated by a truncated Taylor series. In Model II, we acknowledge that this is a Newtonian central force problem and even the

TABLE I. Summary of NGRC models constructed for the magnetic pendulum system. For each model described in Sec. VI, we provide examples of the nonlinear features, their total number (m_{nonlin}), and any additional hyperparameters. (I) Here, $\left(\binom{a}{b}\right)$ denotes the number of ways to choose b items (with replacement) from a set of size a. (II) Here, $\boldsymbol{r}_t = (x_t, y_t)$ are the position coordinates at time t, and $\boldsymbol{c}_i$ is the ith RBF center in 2D, whose x and y coordinates are drawn independently from a uniform distribution over $[-1.5, 1.5]$. (III) Here, $(\tilde{x}_i, \tilde{y}_i)$ are coordinates of the ith magnet in the real system ($i = 1, 2, 3$), and $D(\tilde{x}, \tilde{y})$ is as in Eq. (3).

Model	Nonlinear features	Example term(s)	Addl. hyperparameters	m_{nonlin}
I	Polynomials	$x_t^2\dot{y}_t$ $y_{t-2}\dot{y}_{t-1}\dot{x}_t$	max. degree $d_{\max}$	$\sum_{d=2}^{d_{\max}} \left(\binom{4k}{d}\right)$
II	Radial basis functions	$\dfrac{1}{(\lVert\mathbf{r}_t - \mathbf{c}_i\rVert^2 + h^2)^{3/2}}$	centers $\{\mathbf{c}_i\}_{i=1,\ldots,N_{\text{RBF}}}$	$N_{\text{RBF}}k$
III	Pendulum forces	$\dfrac{\tilde{y}_i - y_t}{D(\tilde{x}_i, \tilde{y}_i)^3}$		$6k$

shape/scale of that force, but plead ignorance about the locations of the point sources. Finally, in Model III, we assume perfect knowledge of the system that generated the time series. Between the linear and nonlinear features, Model III includes all terms in Eqs. (1) and (2).

Our principal question is: How well each NGRC model can reproduce the basins of attraction of the magnetic pendulum and in turn predict its long-term behavior? We focus on the 2D region of initial conditions depicted in Fig. 1, in which the pendulum bob is released from rest at position (x_0, y_0), with $-1.5 \leqslant x_0, y_0 \leqslant 1.5$. We train each model on N_{traj} trajectories generated by Eqs. (1) and (2) from initial conditions sampled uniformly and independently from the same region. We then compare the basins predicted by each trained NGRC model with those of the real system (Appendix D). We define the *error rate* (p) as the fraction of initial conditions for which the basin predictions disagree.

Model I (polynomial features). For NGRC models equipped with polynomial features, excellent training fits can be achieved (Figs. 8, 16, and 17). Despite this, the models struggle to reproduce the qualitative dynamics of the magnetic pendulum, let alone the basins of attraction.

Figure 4(a) shows representative NGRC basin predictions made by Model I using $k = 5$, $d_{\max} = 3$. For the vast majority of initial conditions, the NGRC trajectory does not converge to any of the three attractors, instead diverging to (numerical) infinity in finite time (black points in the middle panels of Fig. 4). Modest improvements can be obtained by including polynomials up to degree $d_{\max} = 5$ (with $k = 3$) as shown in Fig. 4(b). But even here, the model succeeds only at learning the part of each basin in the immediate vicinity of each attractor.

Unfortunately, eking out further improvements by increasing the complexity of the NGRC model becomes computationally prohibitive. When $k = 3$ and $d_{\max} = 5$, for example, the model already has $m = 6188$ features. Likewise, the feature matrix $\mathbf{G}$ used in training has hundreds of millions of entries. With higher values of k and/or $d_{\max}$, the model becomes too expensive to train and simulate on a standard computer.

To ensure the instability of the polynomial NGRC models is not caused by a poor choice of hyperparameters, we have repeated our experiments for a wide range of time resolutions Δt, training trajectory lengths N_{train}, numbers of training trajectories N_{traj} (Fig. 18 in Appendix G), and values for regularization coefficient λ spanning ten orders of magnitude (Fig. 19 in Appendix G). The performance of Model I was not significantly improved in any case.

Model II (radial basis features). For NGRC models using radial basis functions as the readout nonlinearity, the solutions no longer blow up as they did in Model I above. This is encouraging though perhaps unsurprising, as the RBFs are much closer to the nonlinearity in the original equations describing the magnetic pendulum system. Unfortunately, the

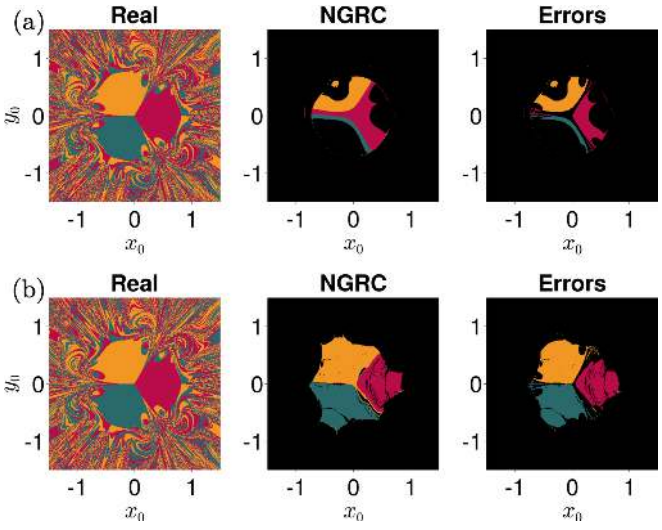

FIG. 4. NGRC models with polynomials as their nonlinearity fail to capture the basins of the magnetic pendulum system. We tested the basin predictions made by NGRC models with the number of time-delayed states up to $k = 5$ and the maximum degree of the polynomial up to $d_{\max} = 5$. Two representative predictions are shown for (a) $k = 5$, $d_{\max} = 3$ and (b) $k = 3$, $d_{\max} = 5$. The left panels show the ground-truth basins of the magnetic pendulum system; The middle panels show the basins identified by the NGRC models, where black points denote initial conditions from which the NGRC trajectories diverge to infinity. The right panels show the correctly identified basins in colors and the misidentified basins in black. The hyperparameters used in this case are $\Delta t = 0.01$, $\lambda = 1$, $N_{\text{traj}} = 100$, and $N_{\text{train}} = 5000$.

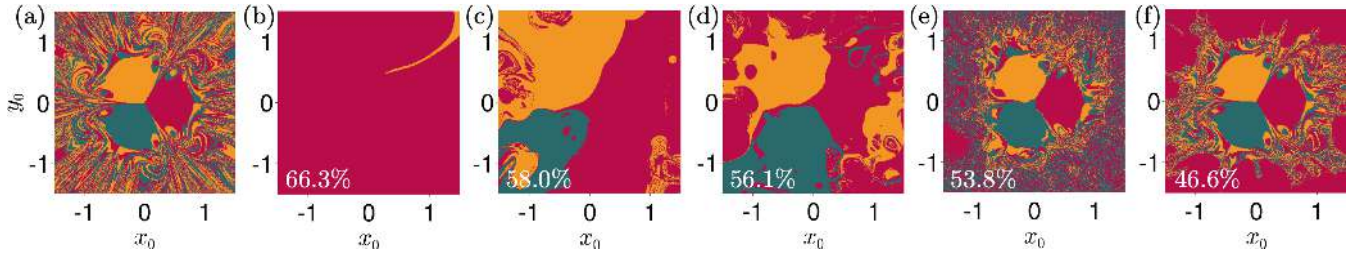

FIG. 5. NGRC models with radial basis functions as their readout nonlinearity struggle to capture the basins of the magnetic pendulum system. We tested the basin predictions made by NGRC models whose nonlinear features include N_{RBF} radial basis functions. Panel (a) shows the ground truth, and the rest of the panels show representative NGRC predictions for (b) $N_{\mathrm{RBF}} = 10$, (c) $N_{\mathrm{RBF}} = 50$, (d) $N_{\mathrm{RBF}} = 100$, (e) $N_{\mathrm{RBF}} = 500$, and (f) $N_{\mathrm{RBF}} = 1000$. The error rates of the predictions are indicated in the lower left corners. The solutions no longer blow up as they did for the polynomial nonlinearities in Model I, but the NGRC models still struggle to capture the basins even qualitatively. Even at $N_{\mathrm{RBF}} = 1000$, only the most prominent features of the basins around the origin are correctly identified. The other hyperparameters used are $\Delta t = 0.01$, $\lambda = 1$, $k = 2$, $N_{\mathrm{traj}} = 100$, and $N_{\mathrm{train}} = 5000$.

accuracy of the NGRC models in predicting basins remains poor.

Figure 5 shows representative NGRC basin predictions as the number of radial basis functions is increased from $N_{\mathrm{RBF}} = 10$ to $N_{\mathrm{RBF}} = 1,000$. In all cases, fits to the training data are impeccable, with the root-mean-square error (RMSE) ranging from 0.003 ($N_{\mathrm{RBF}} = 10$) to 0.0005 ($N_{\mathrm{RBF}} = 1,000$). As more and more RBFs are included, the predictions can be visibly improved, but this improvement is very slow. For example, at $N_{\mathrm{RBF}} = 1,000$ [Fig. 5(f)], the trained model predicts the correct basin for only 53.4% of the initial conditions under study ($p = 0.466$). Moreover, most of this accuracy is attributable to the large central portions of the basins near the attractors, in which the dynamics are closest to linear. Outside of these regions, the NGRC basin map may appear fractal, but the basin predictions themselves are scarcely better than random guesses. This deprives us of accurate forecasts in precisely the regions of the phase space where the outcome is most in question.

As with the polynomial case above, we have repeated our experiments for a wide range of hyperparameters to rule out overfitting or poor model calibration (Figs. 18 and 19 in Appendix G). The accuracy of Model II cannot be meaningfully improved with any of these changes.

Model III (exact nonlinearities). We next test NGRC models equipped with the exact form of the nonlinearity in the magnetic pendulum system, namely the force terms in Eqs. (1) and (2). This time, the NGRC models can perform exceptionally well. Figure 20 (in Appendix G) shows the error rate of NGRC basin predictions as a function of the time resolution Δt. Without any fine-tuning of the other hyperparameters, NGRC models already achieve a near-perfect accuracy of 98.6%, provided Δt is sufficiently small.

Astonishingly, Model III's predictions remain highly accurate even when it is trained on a *single* trajectory ($N_{\mathrm{traj}} = 1$) from a randomly-selected initial condition. Here, NGRC can produce a map of all three basins that is very close to the ground truth (85.0% accuracy, Fig. 6), despite seeing data from only *one* basin during training. This echoes previous results reported for the Li-Sprott system [43], in which NGRC accurately reconstructed the basins of all three attractors (two chaotic, one quasiperiodic) from a single training trajectory. But how can we account for this night-and-day difference with the more system-agnostic models (I and II), which showed poor performance despite 100-fold more training data?

The answer lies in the construction of the NGRC dynamics. In possession of the exact terms in the underlying differential equations, Eq. (10) can—by a suitable choice of the weights $\mathbf{W}$—emulate the action of a numerical integration method from the linear-multistep family [60], whose order depends on k. When $k = 1$, for example, Eq. (10) can mimic an Euler step. Thus, with a sufficiently small step size (Δt), it is not surprising that an NGRC model equipped with exact nonlinearities can accurately reproduce the dynamics of almost any differential equations.

This observation might explain the stellar performance of NGRC in forecasting specific chaotic dynamics like the Lorenz [41] and Li-Sprott systems [43]. The nonlinearities in these systems are quadratic, meaning that so long as $d_{\max} \geqslant 2$, Model I can *exactly* learn the underlying vector field. The only information to be learned is the coefficient ($\mathbf{W}$) that appears before each (non)linear term ($\mathbf{g}$) in the ODEs. This in turn could explain why a single training trajectory suffices to convey information about the phase space as a whole.

Model III with uncertainty. Considering the wide gulf in performance between NGRC models equipped with exact nonlinearity and those equipped with polynomial/radial nonlinearity, it is natural to wonder whether there are some other

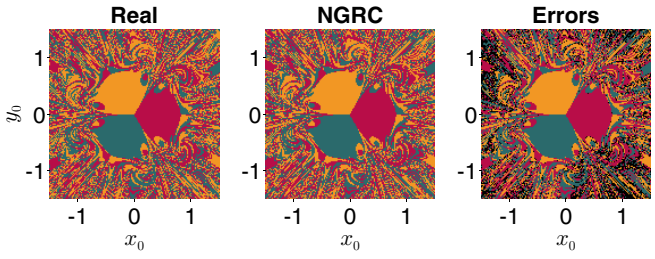

FIG. 6. NGRC models trained on a single trajectory can accurately capture all three basins when the exact nonlinearity from the magnetic pendulum system is adopted. The hyperparameters used are $\Delta t = 0.01$, $\lambda = 10^{-4}$, $k = 4$, $N_{\mathrm{traj}} = 1$, and $N_{\mathrm{train}} = 1000$, which achieves an error rate p of 15%. No systematic optimization was performed to find these parameters. For example, by lowering Δt to 0.0001 and increasing N_{train} to 100 000, we can further improve the accuracy to over 98%.

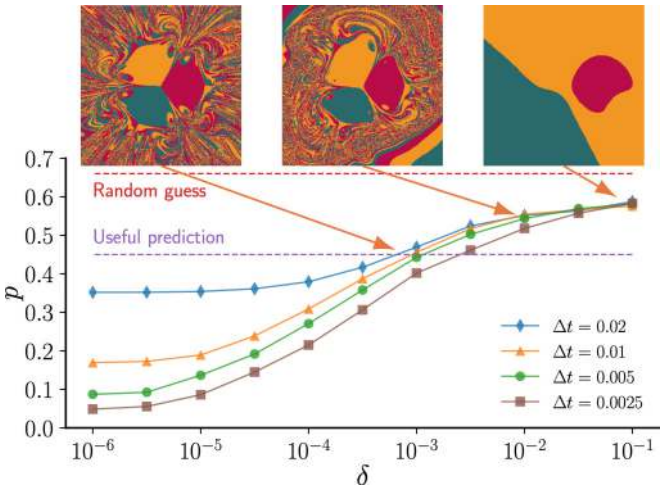

FIG. 7. NGRC basin prediction accuracy when using the exact nonlinearity from the pendulum equations but with small uncertainties. Here, the NGRC models adopt the exact nonlinearity in the magnetic pendulum system, except that the coordinates of the magnets are perturbed by amounts uniformly drawn from $[-\delta, \delta]$. Each data point is obtained by averaging the error rate p over 10 independent realizations. We see that even a small uncertainty on the order of $\delta = 10^{-5}$ can have a noticeable impact on the accuracy of basin predictions. For $\delta > 10^{-2}$, the NGRC predictions become unreliable, approaching the 66.6% failure rate of random guesses. Three representative NGRC-predicted basins are shown for $\delta = 10^{-3}$, $\delta = 10^{-2}$, and $\delta = 10^{-1}$, respectively (all with $\Delta t = 0.01$). We consider predictions with $p < 0.45$ as useful since these in general produce basins that are visually similar to the ground truth. The other hyperparameters used are $\lambda = 1$, $k = 2$, $N_{\text{traj}} = 100$, and $N_{\text{train}} = 5000$.

smart choices of nonlinear features that perform well enough without knowing the exact nonlinearity.

To explore this possibility, we consider a variant of Model III in which we introduce small uncertainties in the nonlinear features, perturbing the assumed coordinates of each magnet by small amounts drawn uniformly and independently between $[-\delta, \delta]$. Here δ is a hyperparameter much smaller than the characteristic spatial scale in this system ($\delta \ll 1$). We train the model on $N_{\text{traj}} = 100$ trajectories from the (unperturbed) real system, then measure how NGRC models perform in the presence of uncertainty about the exact nonlinearity.

In Fig. 7, we see that even a $\sim$1% mismatch ($\delta = 0.01$) in the coordinates of the magnets $(\tilde{x}_i, \tilde{y}_i)$ is enough to make the accuracy of NGRC predictions plunge from almost 100% to below 50% (recall that even random guesses have an accuracy of 33.3%). This extreme sensitivity of NGRC performance to perturbations in the readout nonlinearity suggests that any function other than the exact nonlinearity is unlikely to enable reliable basin predictions in the NGRC model.

Training vs prediction divergence. In all models considered, we have seen that excellent fits to the training data do not guarantee accurate basin predictions for the rest of the phase space. But surprisingly, NGRC models can predict the wrong basin even for the precise initial conditions on which they were trained.

For each of Models I–III, Fig. 8 shows one example training trajectory for which the model attains a near-perfect fit to the ground truth, but the NGRC trajectory from the same initial condition nonetheless goes to a different attractor. We can rationalize this discrepancy by considering the difference between the training and prediction phases as described in Sec. V. During training, NGRC is asked to calculate the next state given the k most recent states from the ground truth data. In contrast, during prediction, the model must make this forecast based on its own (autonomous) trajectory. This permits even tiny errors to compound over time, potentially driving the dynamics to the wrong attractor. Though Fig. 8 shows only one example for each model, these cases are quite common, regardless of the exact hyperparameters used [61].

Moreover, in Fig. 17 in Appendix G, we show that even when the NGRC model predicts the correct attractor for a given training initial condition, the intervening transient dynamics can deviate significantly from the ground truth. This is especially common and pronounced for NGRC models with polynomial or radial nonlinearities [Figs. 17(a) and 17(b)]. In particular, the transient time—how long it takes to come close to the given attractor—can be much larger or smaller than in the real system. As such, reaching the correct attractor does not necessarily imply that an NGRC model has learned the true dynamics from a given training initial condition. To say nothing of the (uncountably many) other initial conditions unseen during training.

Influence of basin complexity. As motivated earlier, the magnetic pendulum is a hard-to-predict system because of its complicated basins of attraction, regardless of the exact parameter values used. And indeed, we see the same sensitivity of NGRC performance to readout nonlinearity for other parameter values, such as $h = 0.3$ and $h = 0.4$ (Fig. 21 in Appendix G).

As the height of the pendulum h is increased, the basins do tend to become less fractal-like. In Fig. 22 in Appendix G, we vary the value of h and show that NGRC models trained with polynomials fail even for the most regular basins ($h = 0.4$). On the other hand, NGRC models trained with radial basis functions see their performance improve significantly as the basins become simpler. As expected, NGRC models equipped with exact nonlinearity successfully capture the basins for all values of h studied.

VII. PREDICTING HIGH-DIMENSIONAL BASINS WITH NGRC

How general are the results presented in Sec. VI? Could the magnetic pendulum be pathological in some unexpected way, with low-order polynomials or other generic features sufficing as the readout nonlinearity for most dynamical systems of interest? To address this possibility, we investigate another paradigmatic multistable system—identical Kuramoto oscillators with nearest-neighbor coupling [62–64],

$$\dot{\theta}_i = \sin(\theta_{i+1} - \theta_i) + \sin(\theta_{i-1} - \theta_i), \quad i = 1, \ldots, n, \quad (14)$$

where we assume a periodic boundary condition, so $\theta_{n+1} = \theta_1$ and $\theta_0 = \theta_n$. Here n is the number of oscillators and hence the dimension of the phase space, and $\theta_i(t) \in [0, 2\pi)$ is the phase of oscillator i at time t.

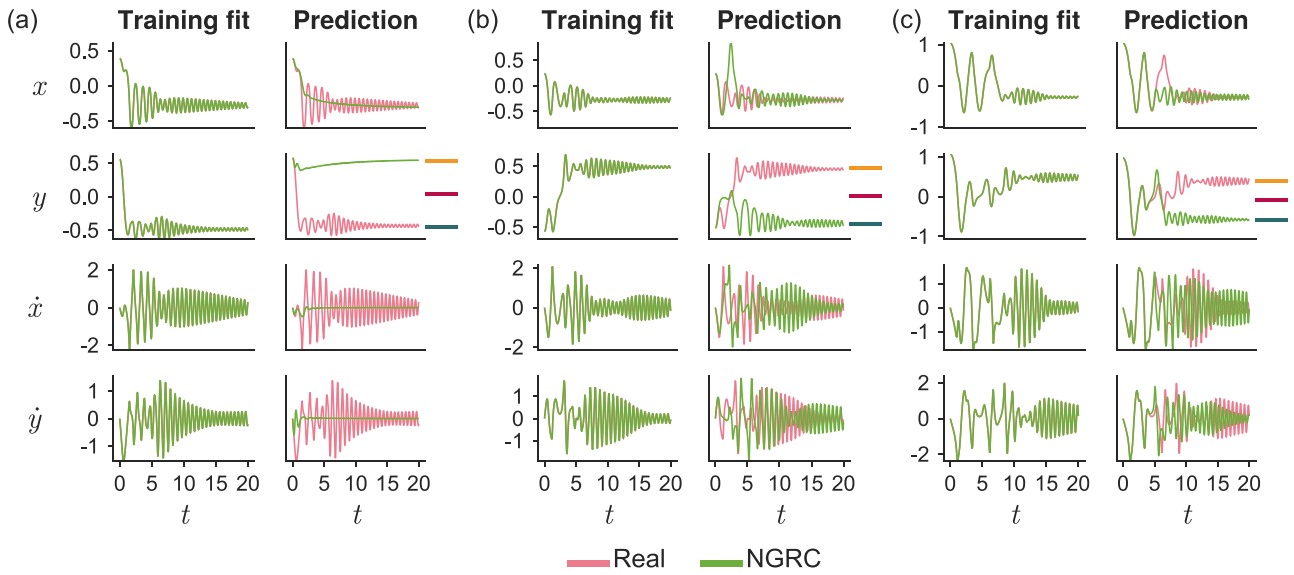

FIG. 8. NGRC models frequently mis-forecast even the initial conditions they were trained on. The panels correspond to (a) Model I, with $d_{\max} = 3$; (b) Model II, with $N_{\text{RBF}} = 500$; and (c) Model III, with no uncertainty. Each model was trained on trajectories from $N_{\text{traj}} = 100$ initial conditions. For each model, we show one such initial condition for which NGRC (green) predicts the wrong basin, despite an excellent fit to the corresponding ground-truth training trajectory (pink). The left column of each panel shows the training fit. The right column shows the autonomous NGRC simulation from the same initial condition. The three magnets can be distinguished by their y coordinates (cf. Fig. 1), allowing us to indicate which one a given trajectory approaches via the three colored lines beside the second row in each panel. In each case, the training fit is impeccable, with the two curves overlapping to within visual resolution (left). Yet when the NGRC model is run *autonomously* from the same initial condition, it quickly diverges from the ground truth, eventually going to an incorrect attractor (right). For all models, we set the other hyperparameters as $\Delta t = 0.01$, $\lambda = 1$, $k = 2$, and $N_{\text{train}} = 5000$.

Aside from being well studied as a model system: the Kuramoto system has two nice features. First, its sine nonlinearities are more "tame" than the algebraic fractions in the magnetic pendulum, helping to untangle whether the sensitive dependence observed in Sec. VI afflicts only specific nonlinearities. Second, we can easily change the dimension of Eq. (14) by varying n, allowing us to test NGRC on high-dimensional basins.

For $n > 4$, Eq. (14) has multiple attractors in the form of *twisted states*—phase-locked configurations in which the oscillators' phases make q full twists around the unit circle, satisfying $\theta_i = 2\pi iq/n + C$. Here q is the winding number of the state [62]. Twisted states are fixed points of Eq. (14) for all q, but only those with $|q| < n/4$ are stable [63]. The corresponding basins of attraction can be highly complicated [64], though not fractal-like as in the magnetic pendulum system.

Similar to Sec. VI, we consider three classes of readout nonlinearities assuming increasing knowledge of the underlying system:

(1) Monomials spanned by the nk oscillator states in $\Theta = \{\boldsymbol{\theta}_t, \boldsymbol{\theta}_{t-1}, \ldots, \boldsymbol{\theta}_{t-k+1}\}$, with degree between 2 and $d_{\max}$.

(2) Trigonometric functions of all scalars in Θ, consisting of $\sin(\ell\theta_i)$ and $\cos(\ell\theta_i)$ for all i and for integers $1 \leqslant \ell \leqslant \ell_{\max}$.

(3) The exact nonlinearity in Eq. (14), namely $\sin(\theta_i - \theta_j)$ for all pairs of connected nodes i and j.

To test the performance of different NGRC models on the Kuramoto system, we first set $n = 9$ and use them to predict basins in a two-dimensional (2D) slice of the phase space. Specifically, we look at slices spanned by $\boldsymbol{\theta}_0 + \alpha_1\boldsymbol{P}_1 + \alpha_2\boldsymbol{P}_2$, $\alpha_i \in (-\pi, \pi]$. Here, $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ are n-dimensional binary orientation vectors, while $\boldsymbol{\theta}_0$ is the base point at the center of the slice.

Figure 9 shows results for orientation vectors given by

$$\boldsymbol{P}_1 = [1, 0, 1, 0, 1, 0, 1, 0, 1], \quad \boldsymbol{P}_2 = [0, 1, 0, 1, 0, 1, 0, 1, 0],$$

with $\boldsymbol{\theta}_0$ representing the two-twist state. We can see that NGRC models with polynomial nonlinearity and trigono-

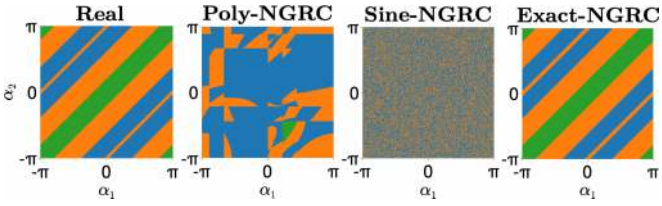

FIG. 9. Predicting basins of a Kuramoto oscillator network with NGRC. We show representative NGRC predictions for basins of $n = 9$ locally-coupled Kuramoto oscillators. Here, we select a 2D slice of the phase space centered at the twisted state with $q = 2$. Basins are color-coded by the absolute winding number $|q|$ of the corresponding attractor (blue: $|q| = 0$; orange: $|q| = 1$; green: $|q| = 2$). Despite the simple geometry of the basins and extensive optimization of hyperparameters, NGRC models with polynomial nonlinearity ($d_{\max} = 2$) or trigonometric nonlinearity ($\ell_{\max} = 5$) have accuracies that are comparable to random guesses. In contrast, with exact nonlinearity, NGRC predictions are consistently over 99% correct. The other hyperparameters are $\Delta t = 0.01$, $\lambda = 10^{-5}$, $k = 2$, $N_{\text{traj}} = 1000$, and $N_{\text{train}} = 3000$.

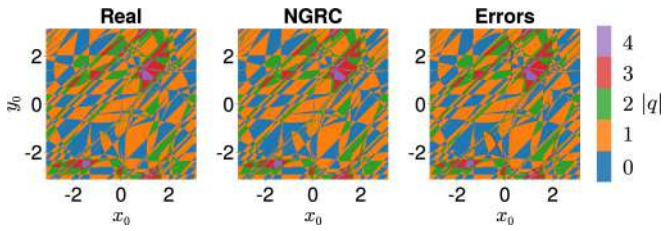

FIG. 10. NGRC with exact nonlinearity can accurately predict high-dimensional basins. Here we train an NGRC model (equipped with exact nonlinearity) to predict the high-dimensional basins of $n = 83$ locally-coupled Kuramoto oscillators. To test the NGRC performance, we randomly select a 2D slice of the 83-dimensional phase space and compare the predicted basins with the ground truth. Basins are color-coded by the absolute winding number $|q|$ of the corresponding attractor. Despite the fragmented and high-dimensional nature of the basins, NGRC captures the intricate basin geometry with ease. Without deliberate optimization of the hyperparameters, NGRC can already achieve over 97% accuracy. The hyperparameters used are $\Delta t = 0.01$, $\lambda = 10^{-5}$, $k = 2$, $N_{\text{traj}} = 1000$, and $N_{\text{train}} = 3000$.

metric nonlinearity fail utterly at capturing the simple ground-truth basins. This is despite an extensive search over the hyperparameters Δt, λ, $d_{\max}$, and $\ell_{\max}$. On the other hand, the NGRC model with exact nonlinearity gives almost perfect predictions for a wide range of hyperparameters. The hyperparameters in Fig. 9 are chosen so that trajectories predicted by the polynomial-NGRC model do not blow up.

Next, we show that the NGRC model with exact nonlinearity can predict basins in much higher dimensions and with more complicated geometries. In Fig. 10, we set $n = 83$ and choose $\boldsymbol{\theta}_0$ to be a random point in the phase space. The n-dimensional binary orientation vectors $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ are constructed by randomly selecting $\lfloor n/2 \rfloor$ components to be 1 and the rest of the components are 0. (The results are not sensitive to the particular realizations of $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$.) Using the same hyperparameters as in Fig. 9, the NGRC model achieves an accuracy of 97.5%. Visually, one would be hard pressed to find any difference between the predicted basins and the ground truth.

VIII. DISCUSSION

When can we claim that a machine learning model like RC has "learned" a dynamical system? One basic requirement is a good training fit, but this is far from sufficient. Many (NG)RC models have extremely low training error, but fail completely during the prediction phase (Fig. 8). A stronger criterion germane to chaotic systems is that the predicted trajectory (beyond the training data) should reproduce the "climate" of the strange attractor, for example replicating the Lyapunov exponents [16]. Here, we propose that the ability to accurately predict basins of attraction is another important test a model must pass before it can be trusted as a proxy of the underlying system. This applies as much to single-attractor systems as it does to multistable ones, as a model might produce spurious attractors not present in the original dynamics [35].

Here, we have shown that there exist commonly-studied systems for which basin prediction presents a steep challenge to leading RC frameworks. In standard RC, the model must be warmed up by an overwhelming majority of the transient dynamics, essentially reaching the attractor before prediction can begin. In contrast, NGRC requires minimal warm-up data but is critically sensitive to the choice of readout nonlinearity, with its ability to make basin predictions contingent on having the exact features in the underlying dynamics. Though these frameworks face very different challenges, each presents a "catch-22": The dynamics cannot be learned unless key information about the system is already known.

The basin prediction problem poses distinct challenges from the problem of forecasting chaotic systems, a test (NG)RC has largely passed with flying colors [2,6,8,9,13,16,17,25,41]. In the latter case, the "climate" of a strange attractor can still be accurately reproduced even after the short-term prediction has failed [16]. It is for this reason that—in the most commonly-used benchmark systems (Lorenz-63, Lorenz-96, Kuramoto-Sivashinsky, etc.)—the transients are often deemed uninteresting and discarded during training. But for multistable systems, to predict which attractor an initial condition will converge to, the transient dynamics are the whole story. Therefore, basin prediction can be even more challenging than forecasting chaos. This is true even in the idealized setting considered here, wherein the attractors are fixed points, and the state of the system is fully observed without noise. As such, we suggest that the magnetic pendulum and Kuramoto systems are ideal benchmarks for data-driven methods aiming to learn multistable nonlinear systems.

It has been established that both standard RC and NGRC are universal approximators, which in appropriate limits can achieve arbitrarily good fits to any system's dynamics [29,33]. But in practice, this is a rather weak guarantee. Unlike many other machine learning tasks, achieving a good fit to the flow of the real system [Eq. (5)] is only the first step; we must ultimately evolve the trained model as a dynamical system in its own right. This can invite a problem of stability, similar to the one faced by numerical integrators. Even when the fit to a system's flow is excellent, the autonomous dynamics of an (NG)RC model can be unstable, causing the prediction to diverge catastrophically from the true solution. How to ensure the stability of a trained (NG)RC model in the general case is a major open problem [54].

There are several exciting directions for future research that follow naturally from our results. First, RC's ability to extract global information about a nonlinear system from local transient trajectories is one of its most powerful assets. Currently, we lack a theory that characterizes conditions under which such extrapolations can be achieved by an RC model. Second, several factors could contribute to the difficulty of basin prediction for RC, including the nonlinearity in the underlying equations, the geometric complexity of the basins, and the nature of the attractors themselves. Can we untangle the effects of these factors? Finally, although standard RC requires relatively long initialization data, it tends to show more robustness towards the choice of nonlinearity (i.e., the reservoir activation function) compared to NGRC. Can we develop a new framework that combines standard RC's robustness with NGRC's efficiency and low data requirement?

RC is elegant, efficient, and powerful; but to usher in a new era of model-free learning of complex dynamical systems [57,65–74], it needs to solve the catch-22 created by its fragile dependence on readout nonlinearity (NGRC) or its reliance on long initialization data for every new initial condition (standard RC).

ACKNOWLEDGMENTS

We thank D. Gauthier, M. Girvan, M. Levine, and A. Haji for insightful discussions. Y.Z. acknowledges support from the Schmidt Science Fellowship and Omidyar Fellowship. S.P.C. acknowledges the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), grant RGPIN-2020-05015.

APPENDIX A: SOFTWARE IMPLEMENTATION

All simulations in this study were performed in Julia. For standard RC (Secs. III and IV), we employ the `ReservoirComputing` package in concert with the `BayesianOptimization` package for hyperparameter optimization. For NGRC (Secs. V–VII), we use a custom implementation as described in Sec. V. Our source code is freely available online [75].

APPENDIX B: NUMERICAL INTEGRATION

For the purpose of obtaining trajectories of the real system for training and validation, we use Julia's `DifferentialEquations` package to integrate all continuous equations of motion (4) using a ninth-order integration scheme (Vern9), with absolute and relative error tolerances both set to 10^{-10}. We stress that the hyperparameter Δt has no relation to the numerical integration step size, which is determined adaptively to achieve the desired error tolerances. Instead, Δt simply represents the timescale at which we seek to model the real dynamics via (NG)RC, and hence the resolution at which we sample the continuous trajectories to generate training and validation data.

APPENDIX C: (NORMALIZED) ROOT-MEAN-SQUARE ERROR

Given an (NG)RC predicted trajectory $\tilde{\mathbf{x}}_t$ and a corresponding trajectory of the real system $\mathbf{x}_t$—each of length N—we calculate the root-mean-square error (RMSE) as

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_t \|\mathbf{x}_t - \tilde{\mathbf{x}}_t\|^2}, \tag{C1}$$

where $\|\cdot\|$ denotes the Euclidean norm. To obtain a normalized version of this (NRMSE)—which we use as part of the objective function to optimize standard RC hyperparameters (Appendix F)—we first rescale each component of $\mathbf{x}_t$ and $\tilde{\mathbf{x}}_t$ by their range in the real system, e.g.,

$$\mathbf{x}_{i,t} \to \frac{\mathbf{x}_{i,t}}{\mathbf{x}_{i,\max} - \mathbf{x}_{i,\min}}, \tag{C2}$$

where the maximum ($\mathbf{x}_{i,\max}$) and minimum ($\mathbf{x}_{i,\min}$) for dimension $i = 1, \ldots, n$ of the state space are calculated over the corresponding training data.

APPENDIX D: BASIN PREDICTION

We associate a given condition $\mathbf{x}_0$ with a basin of attraction by simulating the real (NGRC) dynamics for a total of T time units ($\lceil T/\Delta t \rceil$ iterations). We then identify the closest stable fixed point at the end of the trajectory. In the magnetic pendulum, this is taken as the closest magnet. In the Kuramoto system, we calculate the winding number $|q|$ and use it to identify the corresponding twisted state. We use $T = 100$ for both systems, which is sufficient for all initial conditions under study to approach one of the stable fixed points.

APPENDIX E: SUPPLEMENTAL TABLES

TABLE II. Optimizable hyperparameters in standard RC. Each hyperparameter is optimized in logarithmic scale between the given bounds.

Hyperparameter	Meaning	Lower bound	Upper bound
ρ	Spectral radius of reservoir matrix ($\mathbf{W}$)	10^{-3}	1
s_x	Input scaling (position)	10^{-3}	10
s_v	Input scaling (velocity)	10^{-3}	10
s_b	Bias scaling	10^{-3}	1
α	Leaky coefficient	10^{-3}	1

TABLE III. Values of optimized hyperparameters for standard RC. We list the value of each hyperparameter after optimization to five significant figures.

		Hyperparameter values				
Initial condition (x_0, y_0)	Figures	ρ	s_x	s_v	s_b	α
(–1.3, 0.75)	2 and 3	0.44077	5.5064	0.027882	1.0000	1.0000
(1.0, –0.5)	11 and 12	0.40633	5.0712	0.44366	1.0000	1.0000
(1.75, 1.6)	13 and 14	0.39391	2.9633	0.26557	1.0000	1.0000

APPENDIX F: HYPERPARAMETER OPTIMIZATION

Given an initial condition $\mathbf{x}_0 = (x_0, y_0, \dot{x}_0, \dot{y}_0)^T$ of the magnetic pendulum system, we identify an optimal set of RC hyperparameters using Bayesian optimization. The goal here is to find the minimizer $\mathbf{p}^*$ of a (noisy) function $\mathcal{F}(\mathbf{p})$, i.e.,

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} \ \mathcal{F}(\mathbf{p}). \tag{F1}$$

In our setting, $\mathbf{p} = (\rho, s_x, s_v, s_b, \alpha)^T$ is a vector of our optimizable hyperparameters, and $\mathcal{F}$ is a scalar objective function measuring the error between the real system and a trained RC model generated with those hyperparameters. Typically, this objective function incorporates the NRMSE (Appendix C) between the real and RC-predicted trajectories [30]. But what is the best choice?

We found that the NRMSE during training is a poor optimization objective. In the magnetic pendulum, the resulting RC dynamics tend to blow up during the subsequent autonomous prediction, rather than staying near the fixed point of the real system. Accordingly, we use an objective function that incorporates both training *and* validation NRMSE. Specifically, for a given set of hyperparameters $\mathbf{p}$, we generate one random RC model and train it to the first $N_{\text{train}} = 4000$ steps of the real trajectory starting from $\mathbf{x}_0$. This yields a training NRMSE $\varepsilon_{\text{train}}$. We then simulate the trained RC model for an additional $N_{\text{validation}}$ time steps, picking up where the training left off. This yields a validation NRMSE, $\varepsilon_{\text{validation}}$. We then calculate $\mathcal{F}(\mathbf{p})$ as

$$\mathcal{F}(\mathbf{p}) = \log(\varepsilon_{\text{train}}) + \log(\varepsilon_{\text{validation}}). \tag{F2}$$

We find that this approach yields optimal RC models that have excellent training fits, but remain "well behaved" (i.e., nearly stationary) beyond the training phase.

All hyperparameter optimization for standard RC was performed using the `BayesianOptimization` package in Julia. We model the landscape of $\mathcal{F}$ via Gaussian process regression to observed values of $(\mathbf{p}, \mathcal{F}(\mathbf{p}))$. We employ the default squared-exponential (Gaussian) kernel, with tunable parameters corresponding to the standard deviation plus the length scale of each dimension of the hyperparameter space. We first bootstrap the kernel (fit its parameters) using 200 random sets of hyperparameters $\mathbf{p}$ generated log-uniformly between the bounds in Table II via Latin hypercube sampling. At every step of the process thereafter, we acquire a new candidate value of $\mathbf{p}$ via the commonly-used expected improvement strategy. We repeat this process for a total of 500 iterations, returning the observed minimizer of $\mathcal{F}(\mathbf{p})$. Every 50 iterations, we refit the kernel parameters via maximum *a posteriori* (MAP) estimation. To account for the stochasticity in $\mathcal{F}$ due to $\mathbf{W}$, $\mathbf{W}_{\text{in}}$, and $\mathbf{b}$, we generate 10 realizations of the RC model for each candidate set of hyperparameters $\mathbf{p}$. Thus, over the course of the optimization, we evaluate $\mathcal{F}$ a total of 12 000 times—2000 for the initial bootstrapping period, and an additional 10 000 during the subsequent optimization.

APPENDIX G: SUPPLEMENTAL FIGURES

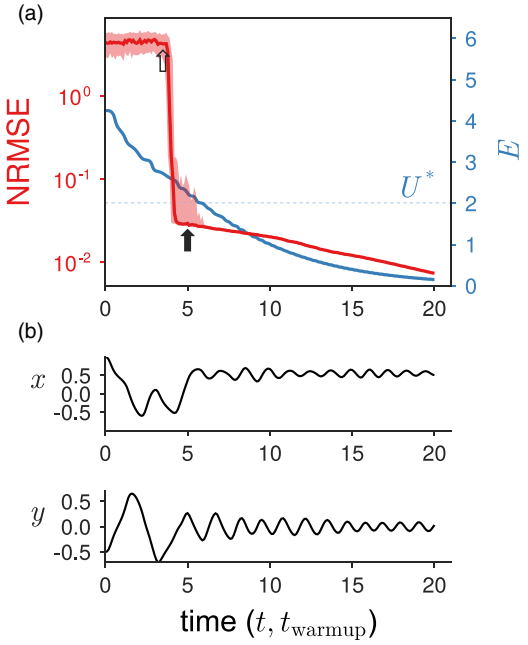

FIG. 11. Forecastability transition of standard RC. Counterpart to Fig. 2 with the initial condition $(x_0, y_0) = (1.0, -0.5)$. The optimized RC hyperparameters for this initial condition are listed in Table III.

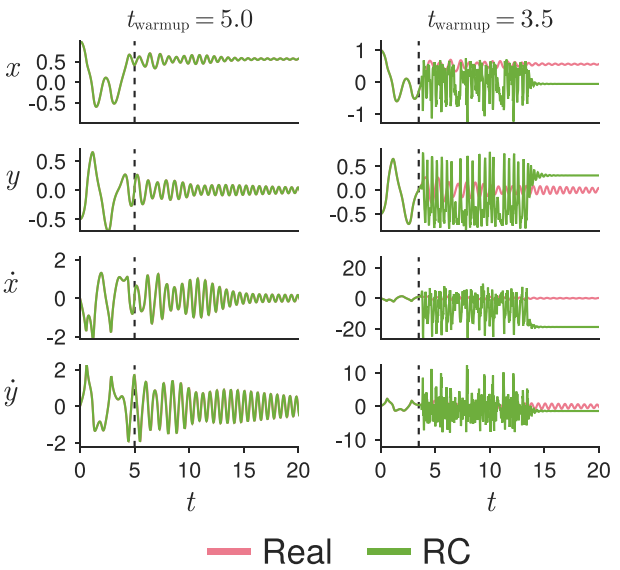

FIG. 12. Sensitivity of standard RC performance to warm-up time. Counterpart to Fig. 3 with the initial condition $(x_0, y_0) = (1.0, -0.5)$. The respective warm-up times indicated by the dashed lines are the same as in Fig. 11.

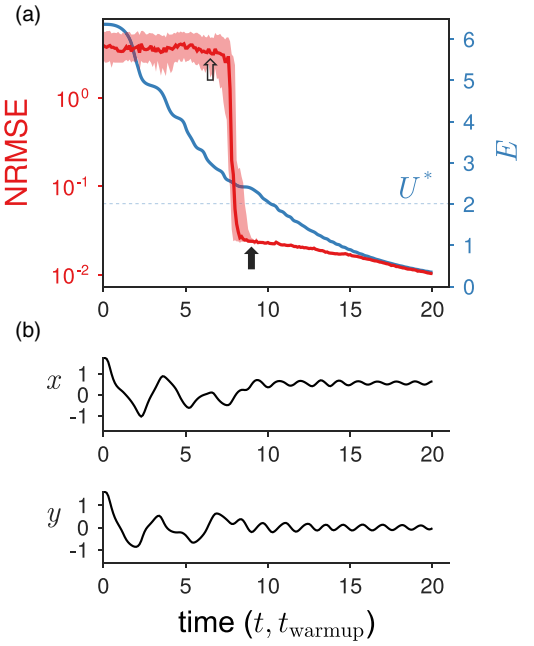

FIG. 13. Forecastability transition of standard RC. Counterpart to Fig. 2 with the initial condition $(x_0, y_0) = (1.75, 1.6)$. The optimized RC hyperparameters for this initial condition are listed in Table III.

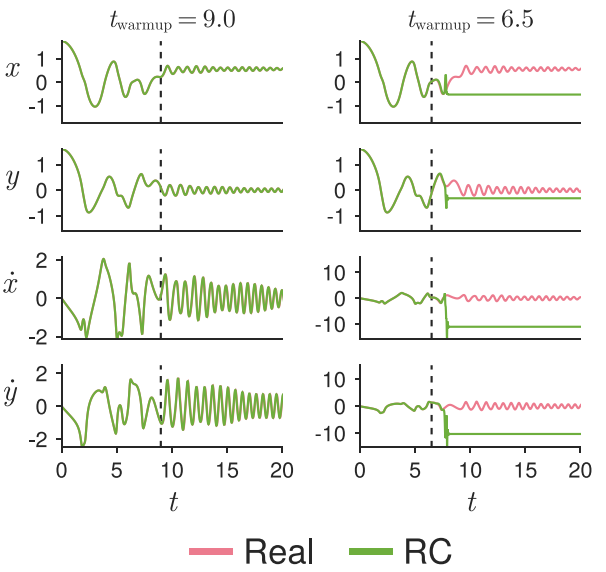

FIG. 14. Sensitivity of standard RC performance to warm-up time. Counterpart to Fig. 3 with the initial condition $(x_0, y_0) = (1.75, 1.6)$. The respective warm-up times indicated by the dashed lines are the same as in Fig. 13.

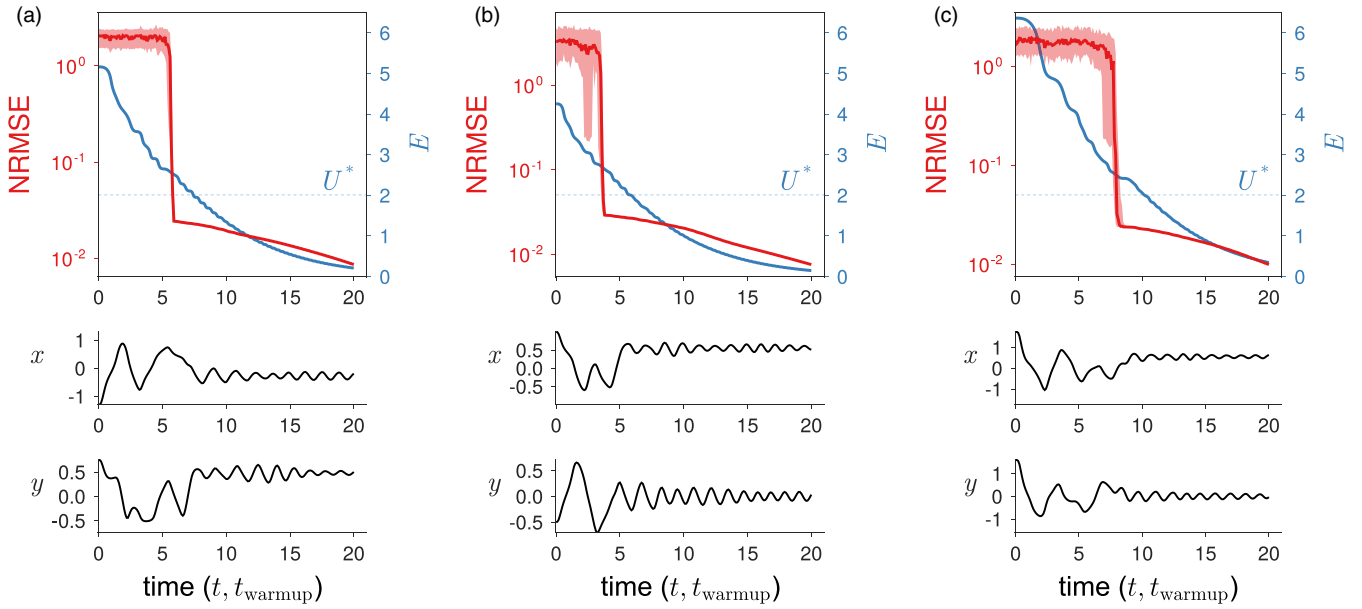

FIG. 15. Forecastability transition of standard RC. Counterparts to Figs. 2, 11, 13 using a larger reservoir size of $N_r = 600$.

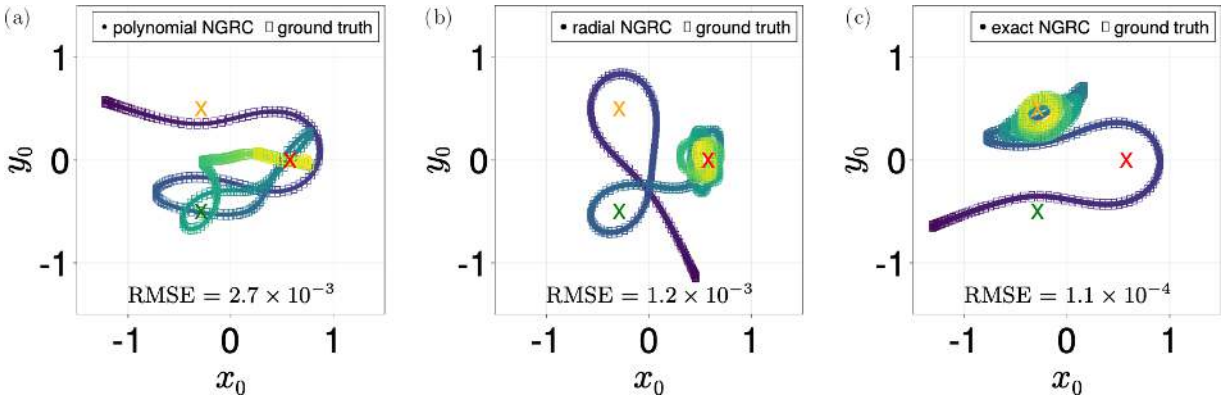

FIG. 16. NGRC models have excellent training fit for all readout nonlinearities tested. Each panel shows an NGRC model with a different readout nonlinearity: (a) polynomials with $d_{\max} = 3$; (b) radial basis functions with $N_{\mathrm{RBF}} = 500$; (c) exact nonlinearity. The trajectories are color-coded in time—they begin with dark purple points and end with bright green points. The three fixed points are represented as crosses. The root-mean-square error (RMSE) for each training trajectory is shown at the bottom of the panel. The other hyperparameters used are $\Delta t = 0.01$, $\lambda = 1$, $k = 2$, $N_{\mathrm{traj}} = 100$, and $N_{\mathrm{train}} = 5000$.

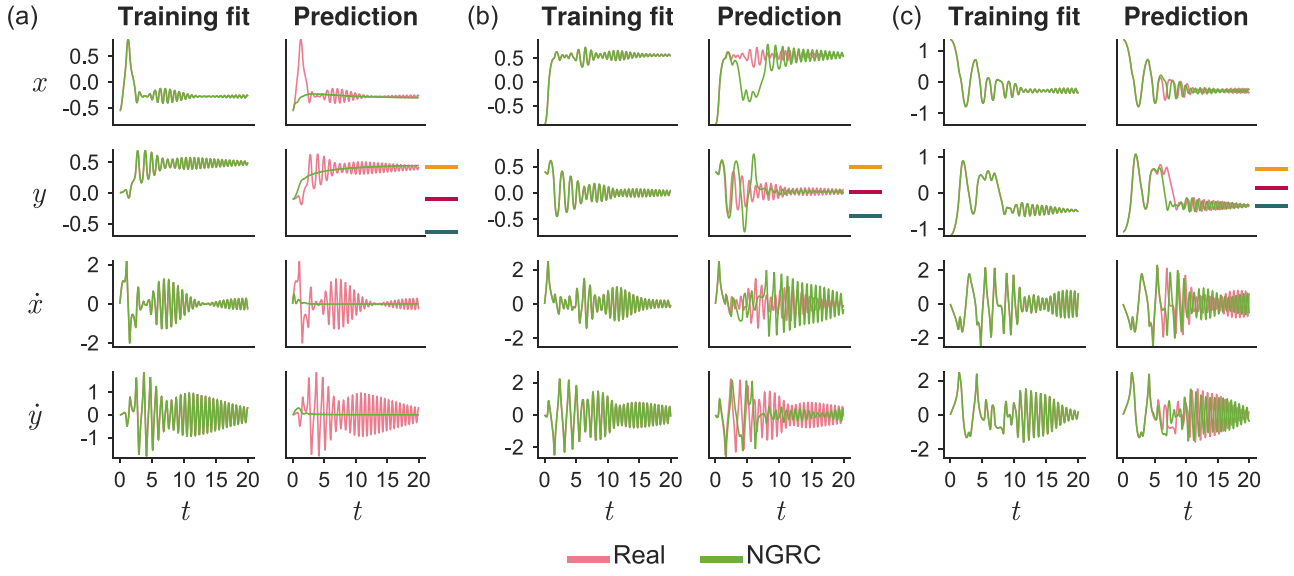

FIG. 17. NGRC models can fail to reproduce the correct transient dynamics even when the attractor is correctly predicted. Counterpart to Fig. 8, showing examples of training initial conditions for which the NGRC predicted trajectory (green, right columns) goes to the correct attractor, but the transient dynamics differs markedly from the ground truth (pink). All hyperparameters are the same as in Fig. 8.

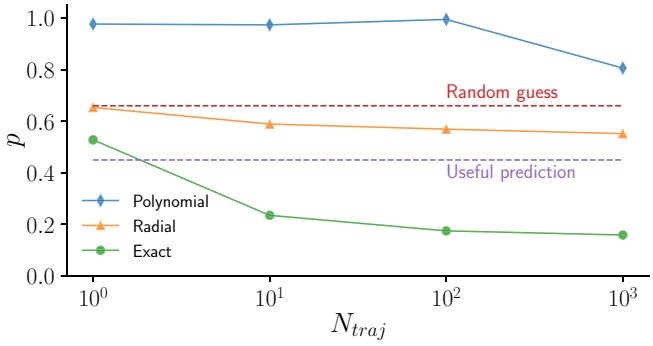

FIG. 18. Error rate p as a function of the number of training trajectories N_{traj} for NGRC models trained with polynomial, radial, and exact nonlinearity. Each data point is obtained by averaging the error rate p over 10 independent realizations. NGRC cannot produce useful predictions with polynomial nonlinearity ($d_{\max} = 3$) or radial nonlinearity ($N_{\mathrm{RBF}} = 100$) no matter how many training trajectories are used. With the exact nonlinearity from the magnetic pendulum equations, NGRC can make accurate predictions once trained on about 10 trajectories. Beyond this, more training trajectories yield only marginal increases in accuracy. The other hyperparameters used here are $\Delta t = 0.01$, $k = 2$, $\lambda = 1$, and $N_{\mathrm{train}} = 5000$.

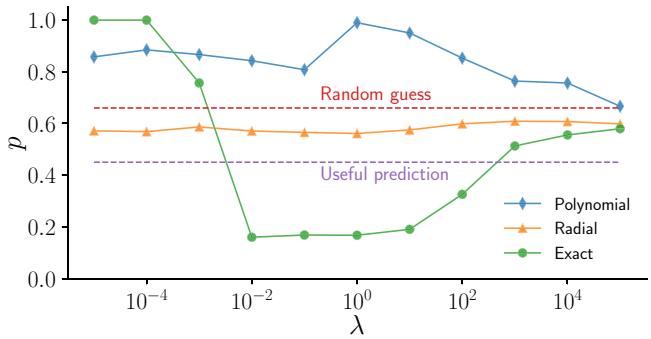

FIG. 19. Error rate p as a function of the regularization coefficient λ for NGRC models trained with polynomial, radial, and exact nonlinearity. Each data point is obtained by averaging the error rate p over 10 independent realizations. No choice of λ can make NGRC produce useful predictions with polynomial nonlinearity ($d_{\max} = 3$) or radial nonlinearity ($N_{\mathrm{RBF}} = 100$). In contrast, exact nonlinearity can produce useful predictions for a wide range of λ (between 10^{-2} and 10^{2}). The other hyperparameters used are $\Delta t = 0.01$, $k = 2$, $N_{\mathrm{traj}} = 100$, and $N_{\mathrm{train}} = 5000$.

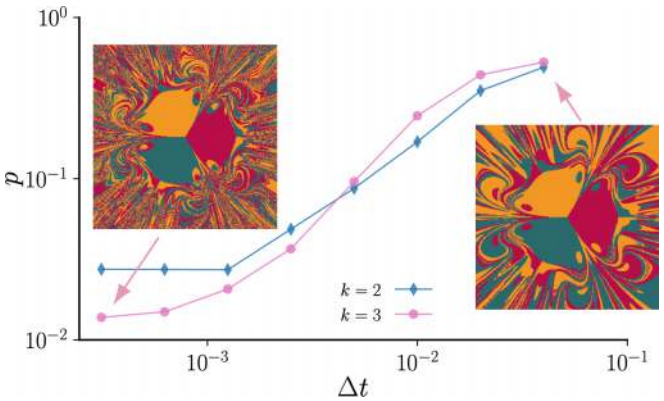

FIG. 20. Dependence of NGRC basin prediction accuracy on the time resolution Δt. Here, the NGRC models adopt the exact nonlinearity in the magnetic pendulum system. Each data point is obtained by averaging the error rate p over 10 independent realizations (error bars are smaller than the size of the symbol). The accuracy of the basin predictions can be significantly improved by taking smaller steps before it plateaus for Δt below a certain threshold. For $\Delta t = 0.0003125$ (the leftmost points) and $k = 3$, the NGRC model consistently achieves an accuracy around 98.6%. Even for $\Delta t = 0.04$ at the other end of the plot (right before NGRC becomes unstable and the solutions blow up), the features of the true basins are qualitatively preserved. Representative NGRC-predicted basins are shown for the two Δt values discussed above. The other hyperparameters used are $\lambda = 1$, $N_{\mathrm{traj}} = 100$, and $N_{\mathrm{train}} = 20\,000$.

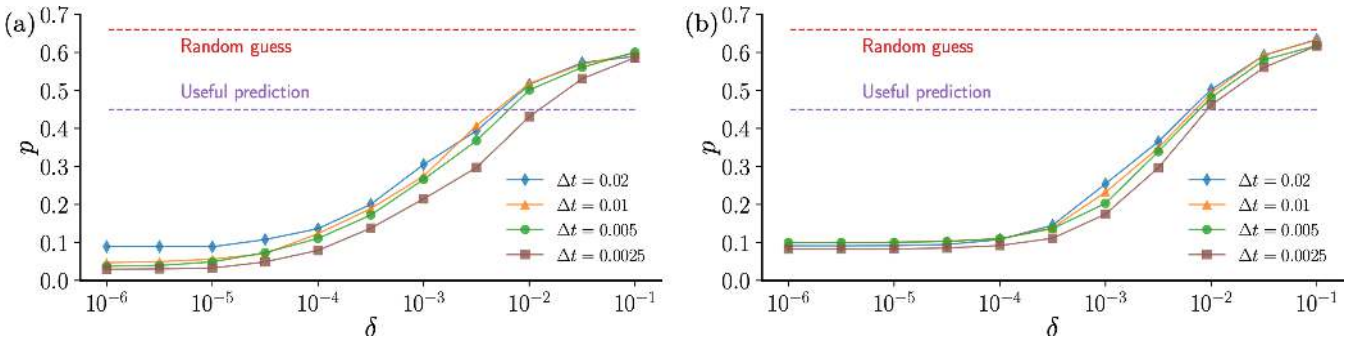

FIG. 21. NGRC basin prediction accuracy when using the exact nonlinearity but with small uncertainties. Same as Fig. 7, but with the height of the pendulum set to (a) $h = 0.3$ and (b) $h = 0.4$.

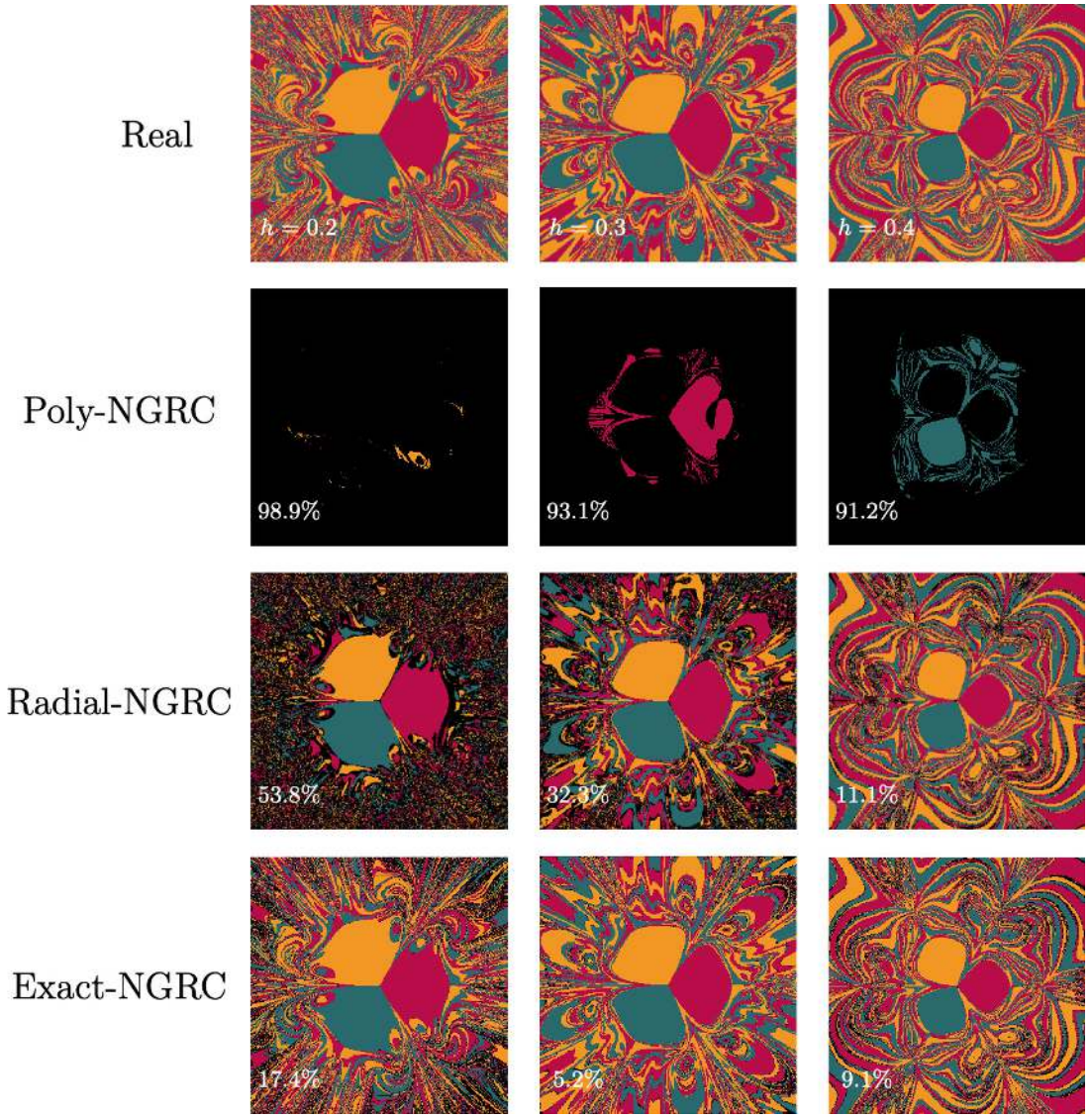

FIG. 22. Basin predictions generally become easier when the basins are less fractal. We show representative NGRC predictions for basins of the magnetic pendulum system with $h = 0.2$, 0.3, and 0.4. As h is increased, the basins become less fractal. For the NGRC predictions, the error rate is marked in white and the wrong predictions are highlighted in black. With polynomial nonlinearity ($d_{\max} = 3$), NGRC predictions are worse than random guesses for all h tested. With radial nonlinearity ($N_{\mathrm{RBF}} = 500$), NGRC predictions become increasingly better as h is increased. With exact nonlinearity, NGRC predictions are consistently good, and the best accuracy is achieved at $h = 0.3$ in this particular case. The other hyperparameters used are $\Delta t = 0.01$, $\lambda = 1$, $k = 2$, $N_{\mathrm{traj}} = 100$, and $N_{\mathrm{train}} = 5000$.

[1] W. Maass, T. Natschläger, and H. Markram, Real-time computing without stable states: A new framework for neural computation based on perturbations, Neural Comput. **14**, 2531 (2002).

[2] H. Jaeger and H. Haas, Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication, Science **304**, 78 (2004).

[3] M. Lukoševičius and H. Jaeger, Reservoir computing approaches to recurrent neural network training, Comput. Sci. Rev. **3**, 127 (2009).

[4] L. Appeltant, M. C. Soriano, G. Van der Sande, J. Danckaert, S. Massar, J. Dambre, B. Schrauwen, C. R. Mirasso, and I. Fischer, Information processing using a single dynamical node as complex system, Nat. Commun. **2**, 468 (2011).

[5] D. Canaday, A. Griffith, and D. J. Gauthier, Rapid time series prediction with a hardware-based reservoir computer, Chaos **28**, 123119 (2018).

[6] T. L. Carroll, Using reservoir computers to distinguish chaotic signals, Phys. Rev. E **98**, 052209 (2018).

[7] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos, Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics, Neural Networks **126**, 191 (2020).

[8] M. Rafayelyan, J. Dong, Y. Tan, F. Krzakala, and S. Gigan, Large-Scale Optical Reservoir Computing for Spatiotemporal Chaotic Systems Prediction, Phys. Rev. X **10**, 041037 (2020).

[9] H. Fan, J. Jiang, C. Zhang, X. Wang, and Y.-C. Lai, Long-term prediction of chaotic systems with machine learning, Phys. Rev. Res. **2**, 012080(R) (2020).

[10] G. A. Gottwald and S. Reich, Combining machine learning and data assimilation to forecast dynamical systems from noisy partial observations, Chaos **31**, 101103 (2021).

[11] Y. Zhong, J. Tang, X. Li, B. Gao, H. Qian, and H. Wu, Dynamic memristor-based reservoir computing for high-efficiency temporal signal processing, Nat. Commun. **12**, 408 (2021).

[12] K. Nakajima and I. Fischer, *Reservoir Computing* (Springer, New York, 2021)

[13] J. Pathak, B. Hunt, M. Girvan, Z. Lu, and E. Ott, Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach, Phys. Rev. Lett. **120**, 024102 (2018).

[14] Z. Lu, B. R. Hunt, and E. Ott, Attractor reconstruction by machine learning, Chaos **28**, 061104 (2018).

[15] L. Grigoryeva, A. Hart, and J.-P. Ortega, Learning strange attractors with reservoir systems, Nonlinearity **36**, 4674 (2023).

[16] J. Pathak, Z. Lu, B. R. Hunt, M. Girvan, and E. Ott, Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data, Chaos **27**, 121102 (2017).

[17] J. Z. Kim, Z. Lu, E. Nozari, G. J. Pappas, and D. S. Bassett, Teaching recurrent neural networks to infer global temporal structure from local examples, Nat. Mach. Intell. **3**, 316 (2021).

[18] A. Röhm, D. J. Gauthier, and I. Fischer, Model-free inference of unseen attractors: Reconstructing phase space features from a single noisy trajectory using reservoir computing, Chaos **31**, 103127 (2021).

[19] M. Roy, S. Mandal, C. Hens, A. Prasad, N. Kuznetsov, and M. D. Shrimali, Model-free prediction of multistability using echo state network, Chaos **32**, 101104 (2022).

[20] T. Arcomano, I. Szunyogh, A. Wikner, J. Pathak, B. R. Hunt, and E. Ott, A hybrid approach to atmospheric modeling that combines machine learning with a physics-based numerical model, J. Adv. Model. Earth Syst. **14**, e2021MS002712 (2022).

[21] P. Antonik, M. Gulina, J. Pauwels, and S. Massar, Using a reservoir computer to learn chaotic attractors, with applications to chaos synchronization and cryptography, Phys. Rev. E **98**, 012215 (2018).

[22] T. Weng, H. Yang, C. Gu, J. Zhang, and M. Small, Synchronization of chaotic systems and their machine-learning models, Phys. Rev. E **99**, 042203 (2019).

[23] H. Fan, L.-W. Kong, Y.-C. Lai, and X. Wang, Anticipating synchronization with machine learning, Phys. Rev. Res. **3**, 023237 (2021).

[24] L.-W. Kong, H.-W. Fan, C. Grebogi, and Y.-C. Lai, Machine learning prediction of critical transition and system collapse, Phys. Rev. Res. **3**, 013090 (2021).

[25] D. Patel and E. Ott, Using machine learning to anticipate tipping points and extrapolate to post-tipping dynamics of non-stationary dynamical systems, Chaos **33**, 023143 (2023).

[26] A. Banerjee, J. D. Hart, R. Roy, and E. Ott, Machine Learning Link Inference of Noisy Delay-Coupled Networks with Opto-electronic Experimental Tests, Phys. Rev. X **11**, 031014 (2021).

[27] T. L. Carroll and L. M. Pecora, Network structure effects in reservoir computers, Chaos **29**, 083130 (2019).

[28] J. Jiang and Y.-C. Lai, Model-free prediction of spatiotemporal dynamical systems with recurrent neural networks: Role of network spectral radius, Phys. Rev. Res. **1**, 033056 (2019).

[29] L. Gonon and J.-P. Ortega, Reservoir computing universality with stochastic inputs, IEEE Trans. Neural Netw. Learning Syst. **31**, 100 (2019).

[30] A. Griffith, A. Pomerance, and D. J. Gauthier, Forecasting chaotic systems with very low connectivity reservoir computers, Chaos **29**, 123108 (2019).

[31] T. L. Carroll, Do reservoir computers work best at the edge of chaos?, Chaos **30**, 121109 (2020).

[32] R. Pyle, N. Jovanovic, D. Subramanian, K. V. Palem, and A. B. Patel, Domain-driven models yield better predictions at lower cost than reservoir computers in lorenz systems, Philos. Trans. R. Soc. A **379**, 20200246 (2021).

[33] A. G. Hart, J. L. Hook, and J. H. Dawes, Echo state networks trained by Tikhonov least squares are L2 (μ) approximators of ergodic dynamical systems, Physica D **421**, 132882 (2021).

[34] J. A. Platt, A. Wong, R. Clark, S. G. Penny, and H. D. Abarbanel, Robust forecasting using predictive generalized synchronization in reservoir computing, Chaos **31**, 123118 (2021).

[35] A. Flynn, V. A. Tsachouridis, and A. Amann, Multifunctionality in a reservoir computer, Chaos **31**, 013125 (2021).

[36] T. L. Carroll, Optimizing memory in reservoir computers, Chaos **32**, 023123 (2022).

[37] J. Pathak, A. Wikner, R. Fussell, S. Chandra, B. R. Hunt, M. Girvan, and E. Ott, Hybrid forecasting of chaotic processes: Using machine learning in conjunction with a knowledge-based model, Chaos **28**, 041101 (2018).

[38] A. Wikner, J. Pathak, B. Hunt, M. Girvan, T. Arcomano, I. Szunyogh, A. Pomerance, and E. Ott, Combining machine learning with knowledge-based modeling for scalable forecasting and subgrid-scale closure of large, complex, spatiotemporal systems, Chaos **30**, 053111 (2020).

[39] K. Srinivasan, N. Coble, J. Hamlin, T. Antonsen, E. Ott, and M. Girvan, Parallel Machine Learning for Forecasting the Dynamics of Complex Networks, Phys. Rev. Lett. **128**, 164101 (2022).

[40] W. A. S. Barbosa, A. Griffith, G. E. Rowlands, L. C. G. Govia, G. J. Ribeill, M.-H. Nguyen, T. A. Ohki, and D. J. Gauthier, Symmetry-aware reservoir computing, Phys. Rev. E **104**, 045307 (2021).

[41] D. J. Gauthier, E. Bollt, A. Griffith, and W. A. Barbosa, Next generation reservoir computing, Nat. Commun. **12**, 5564 (2021).

[42] E. Bollt, On explaining the surprising success of reservoir computing forecaster of chaos? The universal machine learning dynamical system with contrast to VAR and DMD, Chaos **31**, 013108 (2021).

[43] D. J. Gauthier, I. Fischer, and A. Röhm, Learning unseen coexisting attractors, Chaos **32**, 113107 (2022).

[44] J. J. Hopfield, Neural networks and physical systems with emergent collective computational abilities., Proc. Natl. Acad. Sci. USA **79**, 2554 (1982).

[45] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, Visualizing the loss landscape of neural nets, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31 (Curran Associates, Inc., 2018).

[46] A. E. Teschendorff and A. P. Feinberg, Statistical mechanics meets single-cell biology, Nat. Rev. Genet. **22**, 459 (2021).

[47] D. A. Rand, A. Raju, M. Sáez, F. Corson, and E. D. Siggia, Geometry of gene regulatory dynamics, Proc. Natl. Acad. Sci. USA **118**, e2109729118 (2021).

[48] G. Schiebinger, J. Shu, M. Tabaka, B. Cleary, V. Subramanian, A. Solomon, J. Gould, S. Liu, S. Lin, P. Berube *et al.*, Optimal-transport analysis of single-cell gene expression identifies developmental trajectories in reprogramming, Cell **176**, 928 (2019).

[49] M. Sáez, R. Blassberg, E. Camacho-Aguilar, E. D. Siggia, D. A. Rand, and J. Briscoe, Statistically derived geometrical landscapes capture principles of decision-making dynamics during cell fate transitions, Cell Syst. **13**, 12 (2022).

[50] P. J. Menck, J. Heitzig, N. Marwan, and J. Kurths, How basin stability complements the linear-stability paradigm, Nat. Phys. **9**, 89 (2013).

[51] P. J. Menck, J. Heitzig, J. Kurths, and H. Joachim Schellnhuber, How dead ends undermine power grid stability, Nat. Commun. **5**, 3969 (2014).

[52] Note that the warm-up time series is different from the training data and is only used after training has been completed.

[53] A. E. Motter, M. Gruiz, G. Károlyi, and T. Tél, Doubly Transient Chaos: Generic Form of Chaos in Autonomous Dissipative Systems, Phys. Rev. Lett. **111**, 194101 (2013).

[54] M. Lukoševičius, A practical guide to applying echo state networks, in *Neural Networks: Tricks of the Trade* (Springer, Berlin, 2012), pp. 659–686.

[55] S. A. Billings, *Nonlinear System Identification: NARMAX Methods in the Time, Frequency, and Spatio-Temporal Domains* (John Wiley & Sons, Hoboken, NJ, 2013).

[56] L. Jaurigue and K. Lüdge, Connecting reservoir computing with statistical forecasting and deep neural networks, Nat. Commun. **13**, 227 (2022).

[57] S. L. Brunton, J. L. Proctor, and J. N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, Proc. Natl. Acad. Sci. USA **113**, 3932 (2016).

[58] A. Rahimi and B. Recht, Random features for large-scale kernel machines, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 20 (Curran Associates, Inc., 2007).

[59] S. Shahi, F. H. Fenton, and E. M. Cherry, Prediction of chaotic time series using recurrent neural networks and reservoir computing techniques: A comparative study, Mach. Learn. Appl. **8**, 100300 (2022).

[60] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations* (John Wiley & Sons, Hoboken, NJ, 2016).

[61] We did not observe any other attractors other than the three ground-truth fixed points and infinity for all NGRC models considered. The absence of more complicated attractors (compared to RC) is likely due to the simpler architecture of NGRC and the dissipativity of the real dynamics (which NGRC models can learn directly via the linear features).

[62] D. A. Wiley, S. H. Strogatz, and M. Girvan, The size of the sync basin, Chaos **16**, 015103 (2006).

[63] R. Delabays, M. Tyloo, and P. Jacquod, The size of the sync basin revisited, Chaos **27**, 103109 (2017).

[64] Y. Zhang and S. H. Strogatz, Basins with Tentacles, Phys. Rev. Lett. **127**, 194101 (2021).

[65] E. Weinan, A proposal on machine learning via dynamical systems, Commun. Math. Stat. **5**, 1 (2017).

[66] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, Neural ordinary differential equations, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31 (Curran Associates, Inc., 2018), pp. 6572–6583.

[67] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, Fourier neural operator for parametric partial differential equations, arXiv:2010.08895.

[68] R. Guimerà, I. Reichardt, A. Aguilar-Mogas, F. A. Massucci, M. Miranda, J. Pallarès, and M. Sales-Pardo, A Bayesian machine scientist to aid in the solution of challenging scientific problems, Sci. Adv. **6**, eaav6971 (2020).

[69] W. Gilpin, Deep reconstruction of strange attractors from time series, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33 (Curran Associates, Inc., 2020), pp. 204–216.

[70] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, Physics-informed machine learning, Nat. Rev. Phys. **3**, 422 (2021).

[71] N. H. Nelsen and A. M. Stuart, The random feature model for input-output maps between Banach spaces, SIAM J. Sci. Comput. **43**, A3212 (2021).

[72] M. Belkin, Fit without fear: Remarkable mathematical phenomena of deep learning through the prism of interpolation, Acta Numerica **30**, 203 (2021).

[73] M. Levine and A. Stuart, A framework for machine learning of model error in dynamical systems, Commun. Am. Math. Soc. **2**, 283 (2022).

[74] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, Modern Koopman theory for dynamical systems, SIAM Rev. **64**, 229 (2022).

[75] Our source code can be found at https://github.com/spcornelius/RCBasins.